

Implementasi Pemrograman Berorientasi Objek pada Sistem Informasi Service Motor Berbasis Framework Django

PORTOFOLIO UAS TP-PBO2025

1. Identitas Proyek :

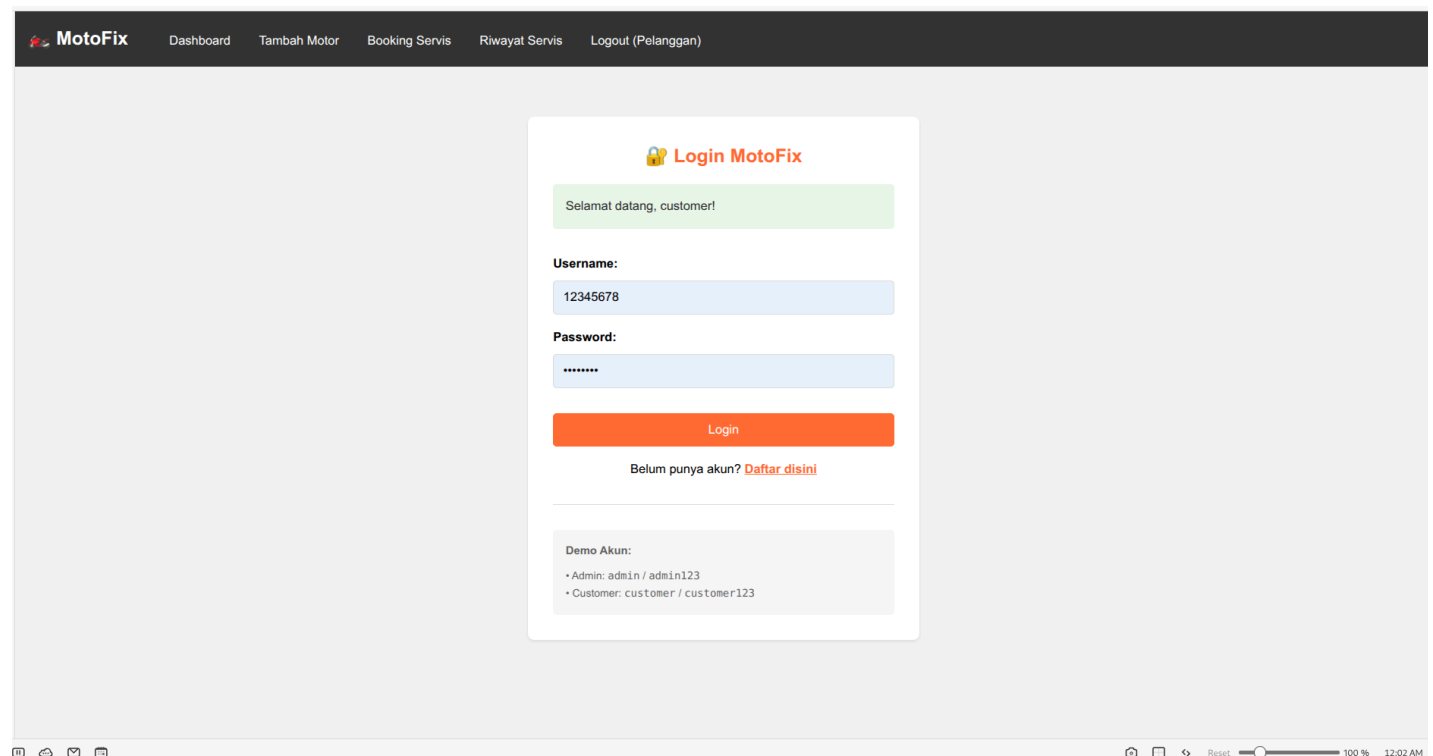
Anggota :

- i. Farhan (2400018009)
- ii. Ahmad Fadhil Fanani (2400018026)
- iii. Dimas Idha Wibowo (2400018049)

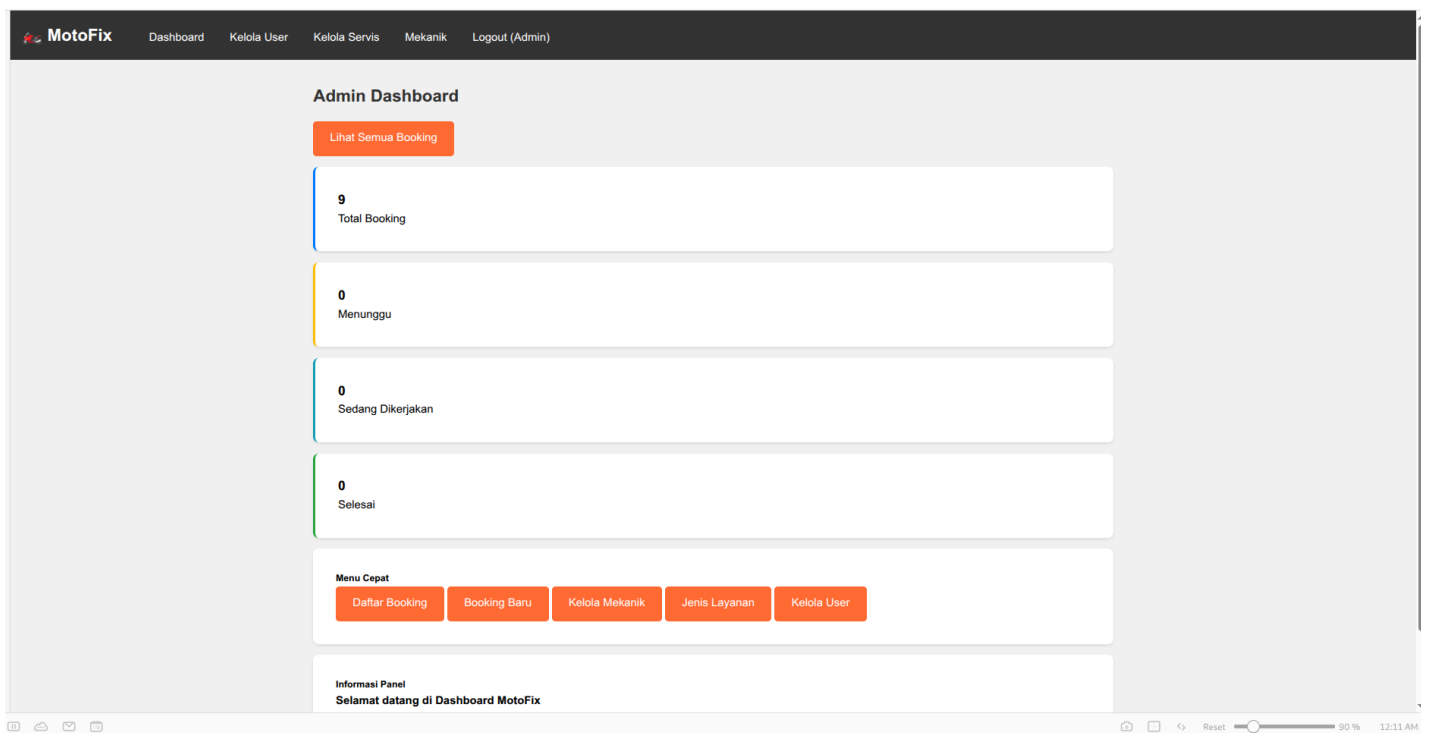
Github : [https://github.com/Revoluz/MotoFix-Web-Based-OOP-Service-Manager-](https://github.com/Revoluz/MotoFix-Web-Based-OOP-Service-Manager-Setup-Guide)

Setup Guide : [README-SETUP.md](#)

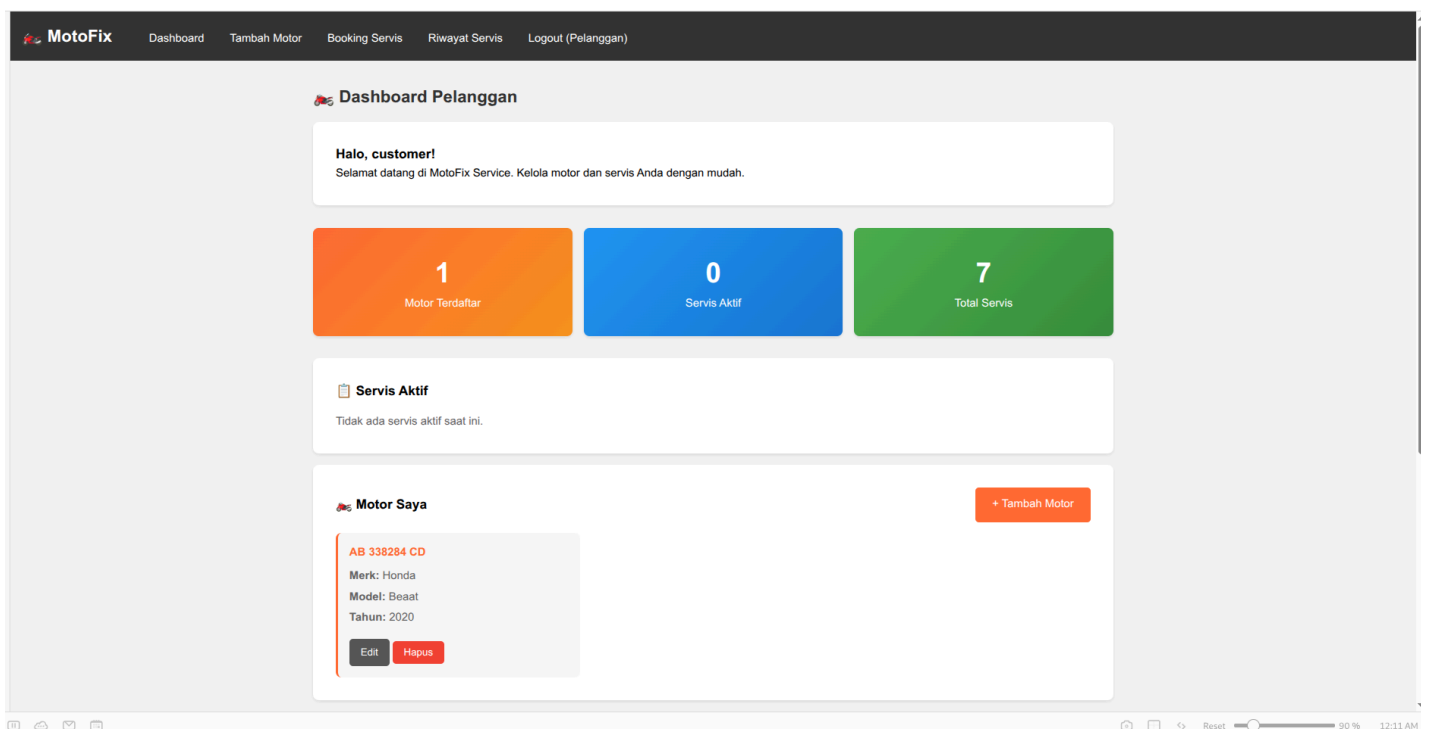
Tampilan Awal Aplikasi :



Tampilan Utama Pegawai/Admin



Tampilan Utama Customer



2. Persoalan Bisnis dan Deskripsi proyek

i. Persoalan Bisnis (Business Problem)

- Berdasarkan analisis terhadap sistem yang berjalan saat ini, teridentifikasi beberapa hambatan operasional utama yang mempengaruhi efisiensi bengkel dan kepuasan pelanggan:
- Inefisiensi Manajemen Data Manual: Proses pendataan pelanggan dan riwayat servis kendaraan saat ini masih dilakukan secara konvensional menggunakan media kertas. Metode ini mengakibatkan tingginya risiko kehilangan data, kesulitan dalam pencarian

riwayat servis pelanggan lama, serta ketidakakuratan dalam pembuatan laporan bulanan karena data tidak tersimpan (backup) secara digital.

- c. Ketidakteraturan Antrian dan Waktu Tunggu: Pelanggan diwajibkan datang langsung ke lokasi untuk melakukan pendaftaran servis, yang sering kali memicu antrean panjang yang tidak teratur. Hal ini menyebabkan ketidakpastian mengenai estimasi waktu pengerjaan bagi pelanggan.
- d. Ketimpangan Beban Kerja Mekanik: Terjadi ketidakseimbangan antara fluktuasi kunjungan pelanggan dengan ketersediaan mekanik di lapangan. Kurangnya data real-time mengenai beban kerja menyebabkan asimetri informasi antara manajemen dan mekanik, yang pada akhirnya menghambat produktivitas operasional.

ii. Deskripsi Proyek (Project Description)

Proyek ini bertujuan untuk mengembangkan sebuah Sistem Informasi Servis Motor Berbasis Web yang mengimplementasikan paradigma Pemrograman Berorientasi Objek (PBO) menggunakan framework Django. Sistem ini dirancang sebagai solusi transformasi digital untuk mengotomatisasi proses bisnis bengkel yang sebelumnya bersifat manual menjadi terintegrasi. Sistem ini mencakup fungsionalitas utama sebagai berikut:

- a. Reservasi Servis Daring
- b. Manajemen Operasional Terpusat
- c. Alokasi Mekanik Dinamis

3. Daftar seluruh spesifikasi aplikasi

Spesifikasi Fungsional

Sistem dirancang dengan fitur-fitur utama untuk mengotomatisasi proses bisnis bengkel:

- i. **Sistem Reservasi Online:** Memungkinkan pelanggan melakukan pendaftaran servis secara mandiri dan memilih jadwal secara *real-time*.
- ii. **Pemantauan Status Reservasi:** Menyajikan informasi status *booking* terkini (menunggu, sedang dikerjakan, atau selesai) kepada pelanggan dan admin.
- iii. **Validasi Reservasi oleh Admin:** Memberikan otoritas kepada admin untuk meninjau, menyetujui, atau menyesuaikan permintaan *booking*.
- iv. **Pengelolaan Penugasan Mekanik:** Memungkinkan admin mengalokasikan tugas kepada mekanik berdasarkan beban kerja dan keahlian.
- v. **Manajemen Daftar Layanan:** Fitur bagi admin untuk memperbarui jenis layanan, harga, dan deskripsi pengerjaan secara dinamis.

Spesifikasi Non-Fungsional

Spesifikasi ini mencakup kebutuhan infrastruktur agar sistem berjalan stabil.

- i. Perangkat Lunak (Software)
 - a. Sistem Operasi: Microsoft Windows 10 atau versi di atasnya.

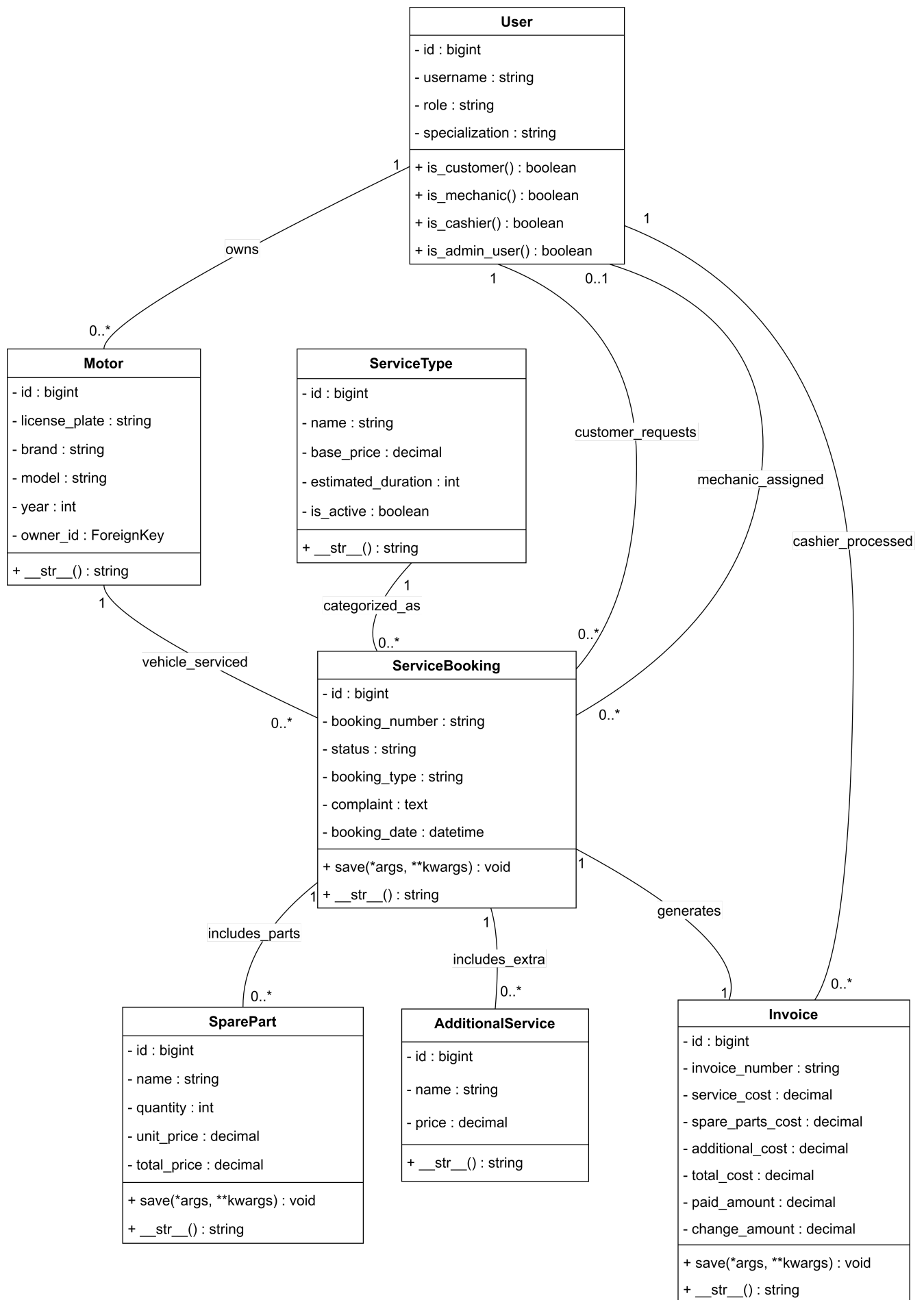
- b. Manajemen Basis Data: MySQL (Relational Database).
- c. Bahasa Pemrograman: Python (sebagai bahasa utama framework Django).
- d. Webserver: XAMPP/Hosting
- e. Alat Perancangan: [Draw.io](https://draw.io) untuk diagram UML.
- f. Text Editor: Visual Studio Code.

ii. Perangkat Keras (Hardware)

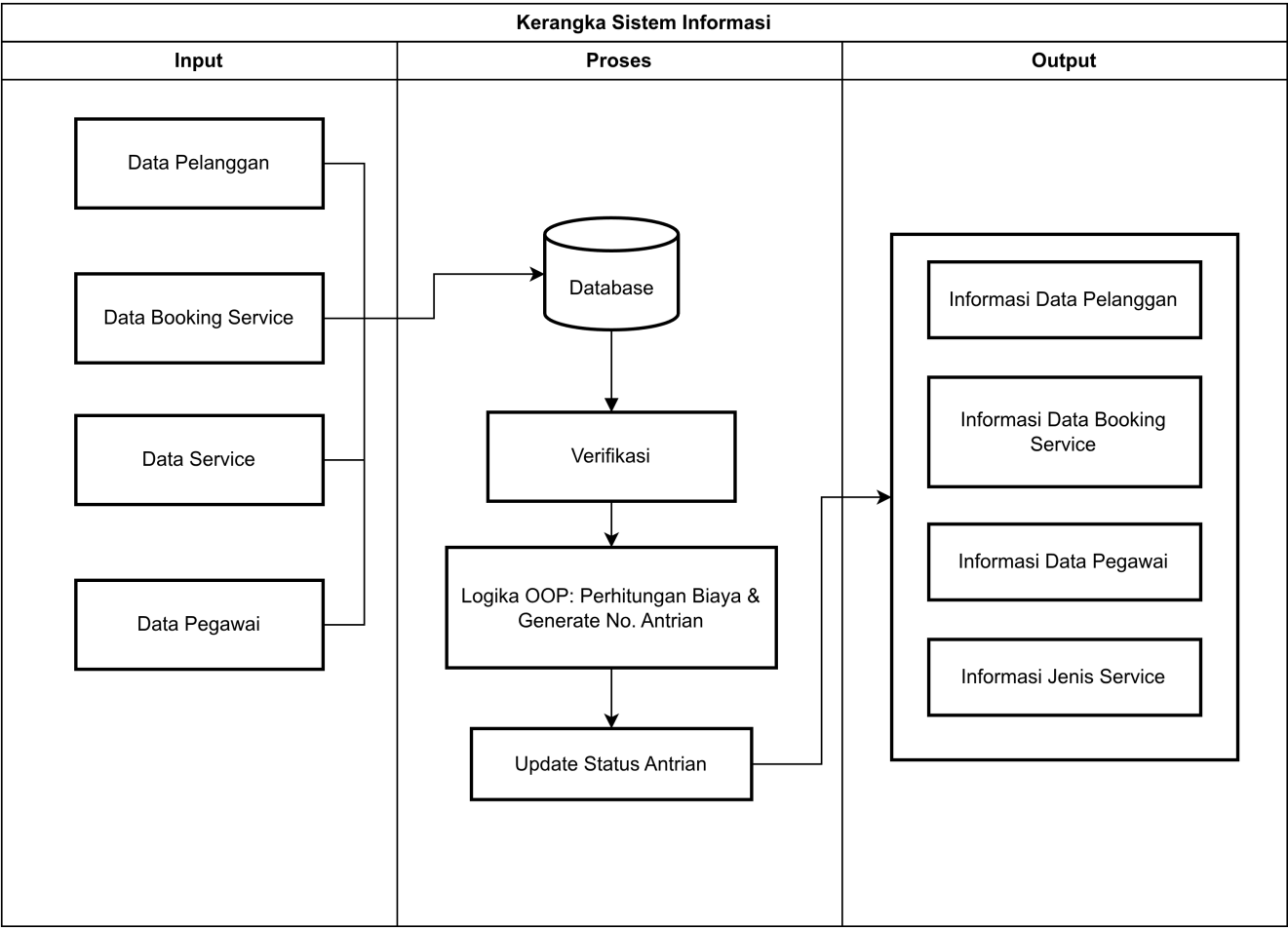
- a. Prosesor: Intel Core i5 Generasi ke-8 (1.6GHz hingga 3.9GHz) atau setara.
- b. Penyimpanan: Harddisk atau SSD dengan kapasitas minimal 500 GB.
- c. Memori (RAM): Kapasitas minimal 4 GB.
- d. Kartu Grafis: Intel UHD Graphics 620 atau standar grafis terintegrasi lainnya.
- e. Perangkat Input: Mouse dan Keyboard.

4. Rancangan Model Diagram UML

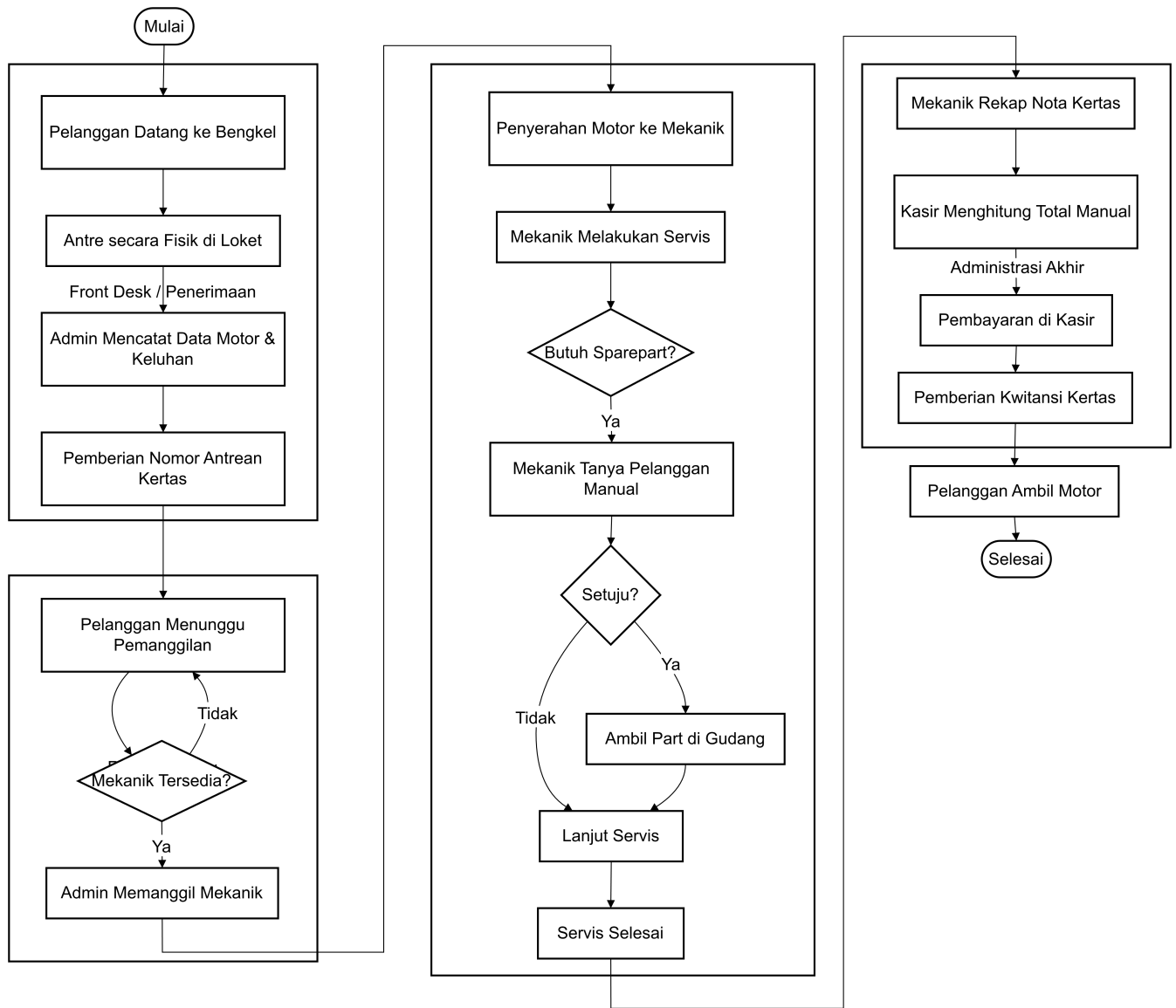
i. UML



ii. Kerangka Sistem Informasi



iii. Flowchart



5. Rancangan Antarmuka berbasis GUI

i. Konsep Desain Antarmuka

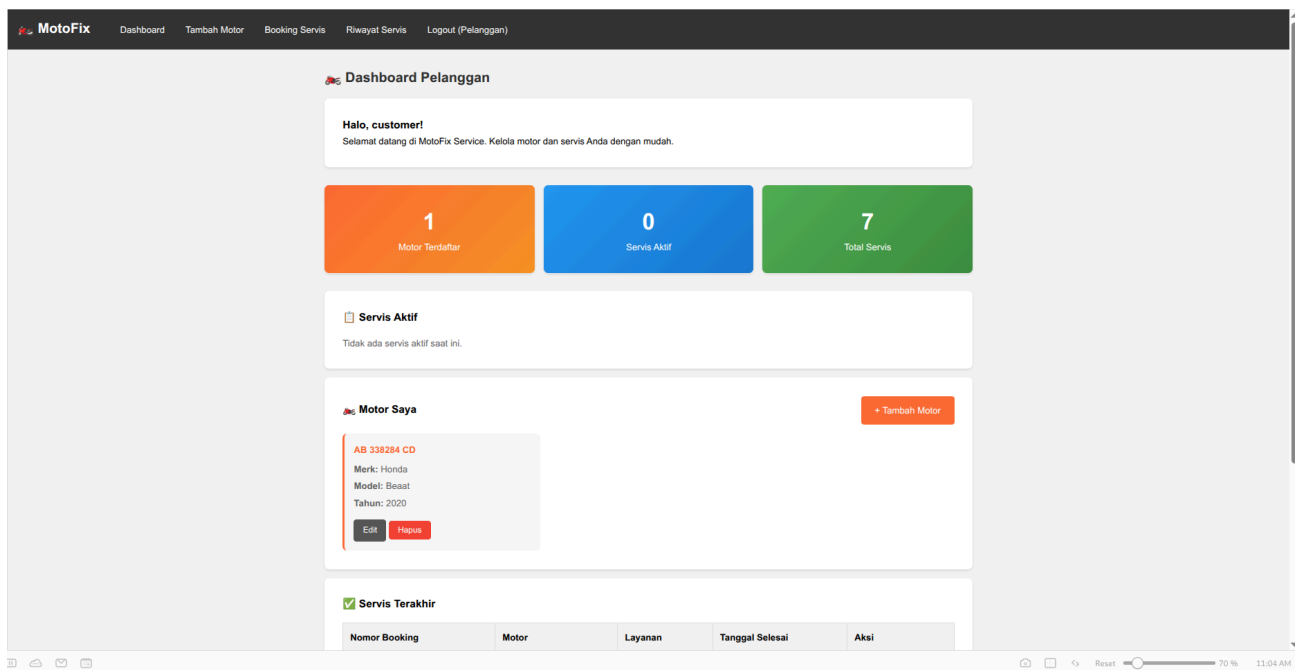
Sistem Informasi Service Motor ini dirancang menggunakan antarmuka berbasis web (Web-Based GUI) yang mengutamakan prinsip User-Friendly dan Responsiveness. Interaksi antara pengguna (Pegawai dan Pelanggan) dengan sistem dilakukan melalui elemen visual seperti formulir input, tombol navigasi, tabel data, dan ikon status, yang bertujuan untuk meminimalisir kesalahan input dan mempercepat proses operasional bengkel

ii. **Implementasi Antarmuka Pengguna** Implementasi rancangan antarmuka dibagi berdasarkan hak akses pengguna, yaitu Pelanggan dan Administrator.

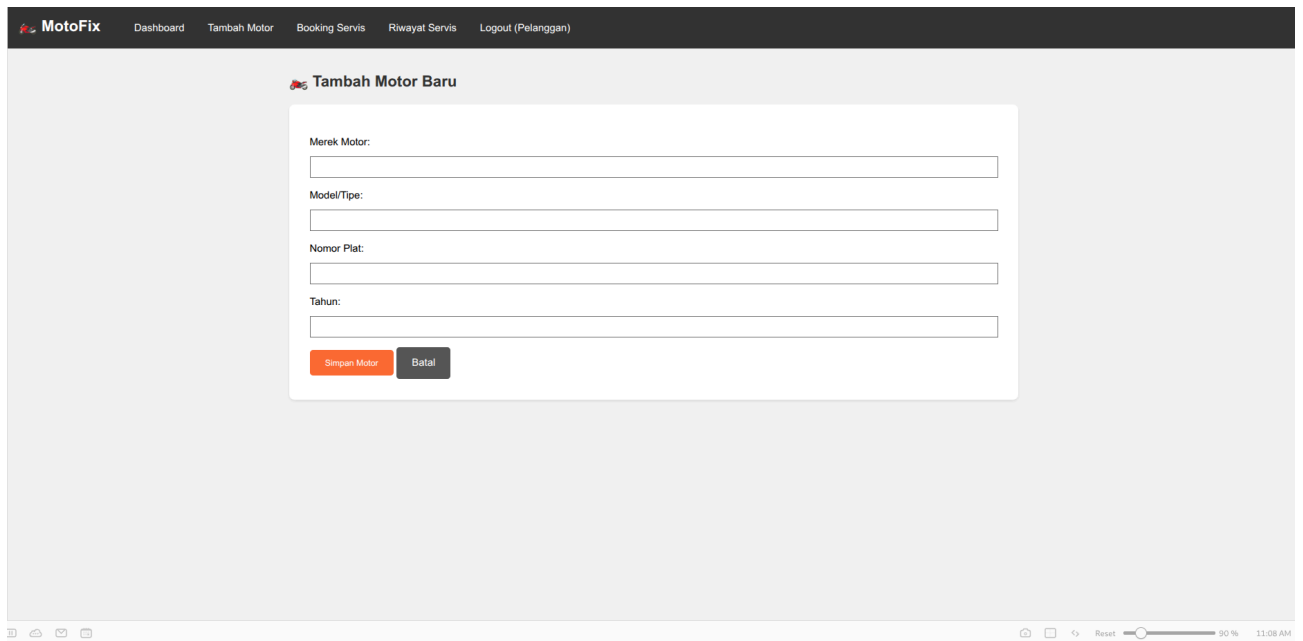
iii. **Halaman Autentikasi (Keamanan Akses)** Sistem menerapkan gerbang keamanan melalui halaman login yang memisahkan akses antara pelanggan dan admin untuk menjaga integritas data.

iv. **Antarmuka Modul Pelanggan (Front-End)** Dirancang dengan tata letak minimalis untuk memudahkan pelanggan awam dalam melakukan reservasi.

a. Dashboard Pelanggan:



b. Manajemen Kendaraan & Booking :



Buat Booking Servis

Nama (opsional): No. Telepon (opsional):

Pilih Motor:

Jenis Layanan:

Keluhan:

Tipe Booking:

[Buat Booking](#) [Batal](#)

c. Riwayat Servis

Riwayat Servis

Filter Status: [Filter](#)

Nomor Booking	Tanggal	Motor	Layanan	Status	Aksi
BK-20260119-9035	19 Jan 2026 11:10	AB 338284 CD Honda Beat	Ganti Ban	Menunggu	Lihat Detail
BK-20260114-5444	14 Jan 2026 21:28	AB 338284 CD Honda Beat	Cuci Motor	Sudah Dibayar	Lihat Invoice
BK-20260114-0005	14 Jan 2026 11:54	AB 338284 CD Honda Beat	Cuci Motor	Sudah Dibayar	Lihat Invoice
BK-20260114-0002	14 Jan 2026 11:45	AB 338284 CD Honda Beat	Ganti Ban	Sudah Dibayar	Lihat Invoice
BK-20260114-0001	14 Jan 2026 11:44	AB 338284 CD Honda Beat	salf	Sudah Dibayar	Lihat Invoice
BK-20251230-7631	30 Des 2025 18:19	AB 338284 CD Honda Beat	Ganti Kampas Rem	Sudah Dibayar	Lihat Invoice
BK-20251230-1603	30 Des 2025 18:19	AB 338284 CD Honda Beat	Cuci Motor	Sudah Dibayar	Lihat Invoice
BK-20251230-8589	30 Des 2025 12:54	AB 338284 CD Honda Beat	Cuci Motor	Sudah Dibayar	Lihat Invoice

[Kembali ke Dashboard](#)

v. Antarmuka Modul Administrator (Back-End) dan Pegawai Dirancang dengan pendekatan Control Panel untuk memberikan kontrol penuh terhadap manajemen operasional bengkel

a. Dashboard Admin :

Manajemen Master Data: Antarmuka CRUD

Kelola User

MotoFix

DashboardKelola UserKelola ServisMekanikLogout (Admin)

Daftar User

Tambah User Baru

ID	Username	Nama	Email	Role	Telepon	Status	Aksi
7	guest082222221768366830863	asda	-	Pelanggan	082222222	Aktif	Detail Edit Hapus
6	guest0811111111768366291168	joko	-	Pelanggan	0811111111	Aktif	Detail Edit Hapus
5	12345678	adf asfas	ahmadbantu3@gmail.com	Kasir	895349381344	Aktif	Detail Edit Hapus
4	admin	sadfas	admin@motofix.com	Admin	None	Aktif	Detail Edit Hapus
3	kasir		kasir@test.com	Kasir	08345678901	Aktif	Detail Edit Hapus
2	mekanik	Rudy	mekanik@test.com	Mekanik	08234567890	Aktif	Detail Edit Hapus
1	customer	Aryo	customer@test.com	Pelanggan	08123456789	Aktif	Detail Edit Hapus

MotoFix

DashboardKelola UserKelola ServisMekanikLogout (Admin)

Edit User

Informasi Akun

Username *

guest0811111111768366291168

Email *

Password (Kosongkan jika tidak diubah)

Konfirmasi Password

Informasi Personal

Nama Depan

joko

Nama Belakang

Role *

Pelanggan

Spesialisasi (Untuk Mekanik)

Umum

Telepon

0811111111

Alamat

None

Akun Aktif

☒

Update User

Batal

MotoFix

DashboardKelola UserKelola ServisMekanikLogout (Admin)

Detail User

Edit

Hapus

Informasi Akun

ID User

7

Username

guest082222221768366830863

Email

-

Role

Pelanggan

Status

Aktif

Informasi Sistem

Terdaftar

14 Jan 2026 12:00

Tersakhir Update

14 Jan 2026 12:00

Login Tersakhir

Belum pernah login

Informasi Personal

Nama Lengkap

asda

Telepon

082222222

Alamat

-

Permissions

Superuser

X Tidak

Staff

X Tidak

Kembali ke Daftar User

Kelola Servis

MotoFixDashboardKelola UserKelola ServismekanikLogout (Admin)

Kelola Jenis Layanan

Tambah Layanan Baru

Daftar Jenis Layanan

Layanan	Detail Harga & Waktu	Status	Aksi
Cuci Motor Pencucian motor dengan shampoo khusus dan semir ban	Harga: Rp 25000 Estimasi: 20 menit	✓ Aktif	Edit Hapus
Ganti Ban Penggantian ban motor (belum termasuk harga ban)	Harga: Rp 50000 Estimasi: 45 menit	✓ Aktif	Edit Hapus
Ganti Kampas Rem Penggantian kampas rem depan/belakang	Harga: Rp 120000 Estimasi: 45 menit	✓ Aktif	Edit Hapus
Ganti Oli Penggantian oli mesin dengan oli berkualitas	Harga: Rp 75000 Estimasi: 30 menit	✓ Aktif	Edit Hapus
Servis Lampu ganti lampu kendaraan	Harga: Rp 22132 Estimasi: 23 menit	✓ Aktif	Edit Hapus
Servis Mesin Perbaikan dan perawatan mesin motor	Harga: Rp 350000 Estimasi: 180 menit	✓ Aktif	Edit Hapus
Servis Rem Perawatan dan perbaikan sistem pengereman	Harga: Rp 100000 Estimasi: 60 menit	✓ Aktif	Edit Hapus
Servis Rutin Servis berkala meliputi ganti oli, cek rem, dan tune up ...	Harga: Rp 150000 Estimasi: 60 menit	✓ Aktif	Edit Hapus
Tune Up Tune up lengkap untuk performa optimal	Harga: Rp 200000 Estimasi: 90 menit	✓ Aktif	Edit Hapus

MotoFixDashboardKelola UserKelola ServismekanikLogout (Admin)

Edit Jenis Layanan

Kembali

Form Edit Layanan

INFORMASI DASAR LAYANAN

Nama Layanan *

Cuci Motor

Nama untuk untuk jenis layanan

Deskripsi

Pencucian motor dengan shampoo khusus dan semir ban

Jelaskan apa yang termasuk dalam layanan ini

HARGA & ESTIMASI WAKTU

Harga Dasar (Rp) *

25000

Harga standar untuk layanan ini

Estimasi Waktu (Menit) *

20

Perkiraan waktu pengerjaan

STATUS LAYANAN

☒ Layanan Aktif

Hanya layanan aktif yang dapat dipilih saat booking

MotoFixDashboardKelola UserKelola ServismekanikLogout (Admin)

25000

Harga standar untuk layanan ini

Estimasi Waktu (Menit) *

20

Perkiraan waktu pengerjaan

STATUS LAYANAN

☒ Layanan Aktif

Hanya layanan aktif yang dapat dipilih saat booking

Update Layanan

Batal

Panduan Pengisian

☒ Nama Layanan

Nama yang jelas dan singkat untuk jenis layanan. Contoh: Servis Berkala, Ganti Oli, Tune-up Engine.

☐ Deskripsi

Jelaskan detail tentang apa yang termasuk dalam layanan ini. Contoh apa saja yang dikerjakan atau diperiksa.

Harga Dasar

Harga standar untuk layanan ini. Bisa disesuaikan per booking sesuai kebutuhan pelanggan.

Estimasi Waktu

Perkiraan waktu pengerjaan dalam satuan menit. Contoh: 30, 60, 120 menit.

Status Aktif

Nonaktifkan layanan jika tidak lagi tersedia. Layanan nonaktif tidak akan muncul di form booking.

MotoFix

DashboardKetola UserKetola ServisMekanikLogout (Admin)

Master Data Mekanik

Daftar Mekanik

Nama	Spesialisasi	No. Telepon	Status	Aksi
Rudy	umum	N/A	AKtif	Edit

Daftar Booking

MotoFix

DashboardKetola UserKetola ServisMekanikLogout (Admin)

Manajemen Booking Service

10

Total Booking

1

Menunggu

0

Ditugaskan

0

Dikerjakan

0

Selesai

8

Dibayar

Filter Status

Semua (10)Menunggu (1)Ditugaskan (0)Sedang Dikerjakan (0)Selesai (0)Dibayar (8)

Daftar Booking

MotoFix

DashboardKetola UserKetola ServisMekanikLogout (Admin)

Daftar Booking

Nomer Booking	Pelanggan	Plat Nomor	Motor	Layanan	Mekanik	Status	Tanggal	Aksi
BK-20260119-9035	Aryo 08123456789	AB 338284 CD	Honda Beaat	Ganti Ban	-	Menunggu	19/01/2026 10:10	Detail
BK-20260114-5444	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	14/01/2026 21:28	Detail
BK-20260114-6006	asda 0822222222	AD 32484 DF	Yamaha MX	Tune Up	Rudy	Dibayar	14/01/2026 12:00	Detail
BK-20260114-6005	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	14/01/2026 11:04	Detail
BK-20260114-6003	joko 08111111111	ABCD	Honda Beaat	Ganti Oli	Rudy	Batal	14/01/2026 11:51	Detail
BK-20260114-6002	Aryo 08123456789	AB 338284 CD	Honda Beaat	Ganti Ban	Rudy	Dibayar	14/01/2026 11:40	Detail
BK-20260114-6001	Aryo 08123456789	AB 338284 CD	Honda Beaat	Servis lampu	Rudy	Dibayar	14/01/2026 11:44	Detail
BK-20251230-7631	Aryo 08123456789	AB 338284 CD	Honda Beaat	Ganti Kampas Rem	Rudy	Dibayar	30/12/2025 18:19	Detail
BK-20251230-1603	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	30/12/2025 18:19	Detail
BK-20251230-8589	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	30/12/2025 12:04	Detail

MotoFixDashboardKelola UserKelola ServisMekanikLogout (Admin)

Detail Booking

BK-20260119-9035

Kembali

Status

Menunggu

Tipe

Booking Online

Pelanggan

Nama

Aryo

Username

customer

Email

customer@test.com

Telepon

08123456789

Alamat

Jl. Pelanggan No. 123, Jakarta

Motor

Plat Nomor: AB 338284 CD

Brand: Honda

Model: Beat

Tahun: 2020

Informasi Layanan

Jenis Layanan: Ganti Ban

Harga Dasar: Rp 50000

Estimasi Durasi: 45 menit

Keluhan & Catatan

Keluhan:

Ban Bocor

Booking Online

Pelanggan

Nama

Aryo

Username

customer

Email

customer@test.com

Telepon

08123456789

Alamat

Jl. Pelanggan No. 123, Jakarta

Motor

Plat Nomor: AB 338284 CD

Brand: Honda

Model: Beat

Tahun: 2020

Informasi Layanan

Jenis Layanan: Ganti Ban

Harga Dasar: Rp 50000

Estimasi Durasi: 45 menit

Keluhan & Catatan

Keluhan:

Ban Bocor

Aksi

Batalkan Booking

Mulai Pengerjaan

Mekanik Ditugaskan

Budi Mekanik

Budi

08123456789

Pelanggan

Nama

Aryo

Username

customer

Email

customer@test.com

Telepon

08123456789

Alamat

Jl. Pelanggan No. 123, Jakarta

Motor

Plat Nomor: AB 338284 CD

Brand: Honda

Model: Beat

Tahun: 2020

Informasi Layanan

Jenis Layanan: Ganti Ban

Harga Dasar: Rp 50000

Estimasi Durasi: 45 menit

Keluhan & Catatan

Keluhan:

Ban Bocor

Aksi

Invoice belum dibuat

Metode Pembayaran: Tunai

Total Tagihan: Rp 50000

Jumlah Uang Dibayar: Masukkan jumlah uang

Kembalian: Rp 0

Buat Invoice & Proses Pembayaran

Mekanik Ditugaskan

Budi Mekanik

Budi

08123456789

Halaman hasil akhir transaksi yang menyajikan rincian biaya (jasa dan suku cadang) secara transparan dan siap cetak sebagai bukti pembayaran sah.

MOTOFIX
Bengkel Motor Profesional

Invoice		Pelanggan	
No Invoice:	INV-BK-20260119-9035	Nama:	Aryo
Tanggal:	19 Januari 2026	Telepon:	08123456789
Kasir:	sadfas	Email:	customer@test.com

Detail Servis		Periode Servis	
Booking:	BK-20260119-9035	Booking:	19/01/2026 11:10
Motor:	Honda Beat	Mulai:	19/01/2026 11:58
Plat Nomor:	AB 338284 CD	Selesai:	19/01/2026 11:58
Mekanik:	Budi Mekanik		

Deskripsi	Qty	Harga	Subtotal
Ganti Ban	1	Rp 50000	Rp 50000
TOTAL PEMBAYARAN:			Rp 50000

Metode Pembayaran: Transfer Bank
Jumlah Dibayar: Rp 50000
Kembalian: Rp 0

[Cetak Invoice](#) [Kembali](#)

Terima kasih atas kepercayaan Anda!
Invoice ini sah dan diproses oleh sistem MotoFix
Dicetak pada: 19 Januari 2026 12:12

vi. Karakteristik GUI yang Dibangun

- Konsistensi Visual:** Penggunaan skema warna, *font*, dan tata letak tombol yang seragam di setiap halaman untuk kenyamanan visual.
- Feedback Sistem:** Antarmuka memberikan umpan balik visual (seperti perubahan status "Menunggu" ke "Selesai") secara *real-time* kepada pengguna.
- Efisiensi Input:** Penggunaan elemen *dropdown* dan *date picker* pada formulir *booking* mengurangi risiko kesalahan pengetikan data (human error).

6. Syntax Program dan penjelasannya

Nama File: `customer/models.py` **Kode Program:**

```

import django.db.models.deletion
import django.utils.timezone
from django.conf import settings
from django.db import migrations, models

class Migration(migrations.Migration):

    initial = True

    dependencies = [
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]

    operations = [
        migrations.CreateModel(
            name='ServiceType',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serializable=True)),
                ('name', models.CharField(max_length=100, verbose_name='Nama Layanan')),
                ('description', models.TextField(blank=True, null=True, verbose_name='Deskripsi')),
                ('base_price', models.DecimalField(decimal_places=2, max_digits=10, verbose_name='Harga Dasar')),
                ('estimated_duration', models.IntegerField(help_text='Durasi estimasi', verbose_name='Durasi Estimasi')),
                ('is_active', models.BooleanField(default=True, verbose_name='Aktif')),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
            ],
            options={
                'verbose_name': 'Jenis Layanan',
                'verbose_name_plural': 'Jenis Layanan',
                'db_table': 'service_types',
                'ordering': ['name'],
            },
        ),
        migrations.CreateModel(
            name='Motor',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serializable=True)),
                ('license_plate', models.CharField(max_length=15, unique=True, verbose_name='Nomor Plat')),
                ('brand', models.CharField(max_length=50, verbose_name='Merk')),
                ('model', models.CharField(max_length=50, verbose_name='Model/Tipe')),
                ('year', models.IntegerField(verbose_name='Tahun')),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
            ],
        ),
    ]

```

```

        ('owner', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE)),
    ],
    options={
        'verbose_name': 'Motor',
        'verbose_name_plural': 'Motor',
        'db_table': 'motors',
        'ordering': ['-created_at'],
    },
),
migrations.CreateModel(
    name='ServiceBooking',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True, serializable=True)),
        ('booking_number', models.CharField(max_length=20, unique=True, verbose_name='Nomor Booking')),
        ('booking_type', models.CharField(choices=[('booking', 'Booking Online'), ('walk_in', 'Walk In')], max_length=20, verbose_name='Tipe Booking')),
        ('status', models.CharField(choices=[('pending', 'Menunggu'), ('assigned', 'Ditugaskan'), ('completed', 'Selesai')], max_length=20, verbose_name='Status')),
        ('complaint', models.TextField(verbose_name='Keluhan/Permintaan')),
        ('notes', models.TextField(blank=True, null=True, verbose_name='Catatan')),
        ('booking_date', models.DateTimeField(default=django.utils.timezone.now, verbose_name='Tanggal Booking')),
        ('assigned_date', models.DateTimeField(blank=True, null=True, verbose_name='Tanggal Ditugaskan')),
        ('start_date', models.DateTimeField(blank=True, null=True, verbose_name='Tanggal Mulai')),
        ('finish_date', models.DateTimeField(blank=True, null=True, verbose_name='Tanggal Selesai')),
        ('payment_date', models.DateTimeField(blank=True, null=True, verbose_name='Tanggal Pembayaran')),
        ('created_at', models.DateTimeField(auto_now_add=True)),
        ('updated_at', models.DateTimeField(auto_now=True)),
        ('customer', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='customer.Customer')),
        ('mechanic', models.ForeignKey(blank=True, limit_choices_to={'role': 'mechanic'}, to='customer.Customer')),
        ('motor', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='motor.Motor')),
        ('service_type', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='service.Service')),
    ],
    options={
        'verbose_name': 'Booking Servis',
        'verbose_name_plural': 'Booking Servis',
        'db_table': 'service_bookings',
        'ordering': ['-booking_date'],
    },
),
migrations.CreateModel(
    name='Invoice',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True, serializable=True)),
        ('invoice_number', models.CharField(max_length=20, unique=True, verbose_name='Nomor Invoice')),
        ('service_cost', models.DecimalField(decimal_places=2, max_digits=10, verbose_name='Biaya Servis')),
    ],
    options={
        'verbose_name': 'Invoice',
        'verbose_name_plural': 'Invoice',
        'db_table': 'invoices',
        'ordering': ['-created_at'],
    },
),
migrations.CreateModel(
    name='Payment',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True, serializable=True)),
        ('payment_number', models.CharField(max_length=20, unique=True, verbose_name='Nomor Pembayaran')),
        ('amount', models.DecimalField(decimal_places=2, max_digits=10, verbose_name='Jumlah Pembayaran')),
        ('due_date', models.DateTimeField(verbose_name='Tanggal Jatuh Tempo')),
        ('paid_date', models.DateTimeField(verbose_name='Tanggal Dibayar')),
        ('customer', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='customer.Customer')),
        ('motor', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='motor.Motor')),
        ('service_type', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='service.Service')),
    ],
    options={
        'verbose_name': 'Pembayaran',
        'verbose_name_plural': 'Pembayaran',
        'db_table': 'payments',
        'ordering': ['-created_at'],
    },
),
migrations.CreateModel(
    name='Appointment',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True, serializable=True)),
        ('appointment_number', models.CharField(max_length=20, unique=True, verbose_name='Nomor Appointment')),
        ('customer', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='customer.Customer')),
        ('motor', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='motor.Motor')),
        ('service_type', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, to='service.Service')),
        ('appointment_date', models.DateTimeField(verbose_name='Tanggal Appointment')),
        ('status', models.CharField(choices=[('pending', 'Menunggu'), ('confirmed', 'Konfirmasi'), ('cancelled', 'Dibatalkan')], max_length=20, verbose_name='Status')),
    ],
    options={
        'verbose_name': 'Appointment',
        'verbose_name_plural': 'Appointment',
        'db_table': 'appointments',
        'ordering': ['-appointment_date'],
    },
),
]


```



```

        ('spare_parts_cost', models.DecimalField(decimal_places=2, default=0,
        ('additional_cost', models.DecimalField(decimal_places=2, default=0,
        ('total_cost', models.DecimalField(decimal_places=2, max_digits=10, ve
        ('payment_method', models.CharField(choices=[('cash', 'Tunai'), ('deb
        ('paid_amount', models.DecimalField(decimal_places=2, max_digits=10,
        ('change_amount', models.DecimalField(decimal_places=2, default=0, ma
        ('notes', models.TextField(blank=True, null=True, verbose_name='Catata
        ('created_at', models.DateTimeField(auto_now_add=True)),
        ('cashier', models.ForeignKey(limit_choices_to={'role': 'cashier'}, on
        ('service_booking', models.OneToOneField(on_delete=django.db.models.de

    ],
    options={
        'verbose_name': 'Invoice',
        'verbose_name_plural': 'Invoice',
        'db_table': 'invoices',
        'ordering': ['-created_at'],
    },
),
migrations.CreateModel(
    name='AdditionalService',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True, serial
        ('name', models.CharField(max_length=100, verbose_name='Nama Jasa')),
        ('description', models.TextField(blank=True, null=True, verbose_name=
        ('price', models.DecimalField(decimal_places=2, max_digits=10, verbose
        ('created_at', models.DateTimeField(auto_now_add=True)),
        ('service_booking', models.ForeignKey(on_delete=django.db.models.dele
    ],
    options={
        'verbose_name': 'Jasa Tambahan',
        'verbose_name_plural': 'Jasa Tambahan',
        'db_table': 'additional_services',
    },
),
migrations.CreateModel(
    name='SparePart',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True, serial
        ('name', models.CharField(max_length=100, verbose_name='Nama Suku Cad
        ('quantity', models.IntegerField(default=1, verbose_name='Jumlah')),
        ('unit_price', models.DecimalField(decimal_places=2, max_digits=10, ve
        ('total_price', models.DecimalField(decimal_places=2, max_digits=10,
        ('notes', models.TextField(blank=True, null=True, verbose_name='Catata

```

```

        ('created_at', models.DateTimeField(auto_now_add=True)),
        ('service_booking', models.ForeignKey(on_delete=django.db.models.dele
    ],
    options={
        'verbose_name': 'Suku Cadang',
        'verbose_name_plural': 'Suku Cadang',
        'db_table': 'spare_parts',
    },
),
]

```

Implementasi Model Basis Data ([models.py](#))

Pada tahap ini, saya mengimplementasikan struktur data dan logika bisnis sistem menggunakan konsep Pemrograman Berorientasi Objek (OOP). Setiap tabel dalam basis data direpresentasikan sebagai sebuah Class (Kelas) yang mewarisi sifat-sifat dasar dari [django.db.models.Model](#).

i. Kelas Motor (Entitas Kendaraan)

Kelas ini berfungsi sebagai cetak biru (blueprint) untuk menyimpan data kendaraan pelanggan.

Relasi: Saya menggunakan ForeignKey untuk menghubungkan motor dengan pemiliknya (owner). Atribut on_delete=models.CASCADE diterapkan agar jika akun pemilik dihapus, data motor yang terkait juga ikut terhapus demi integritas data.

Atribut: Menyimpan spesifikasi fisik kendaraan seperti license_plate (bersifat unique), brand, model, dan year.

ii. Kelas ServiceBooking (Entitas Transaksi Inti)

Ini adalah kelas paling krusial yang menjadi pusat integrasi sistem. Kelas ini menghubungkan Pelanggan, Motor, Mekanik, dan Jenis Layanan.

Manajemen Status: Saya mendefinisikan STATUS_CHOICES untuk memantau siklus hidup servis, mulai dari pending, assigned, in_progress, hingga paid.

Logika Polimorfisme (Overriding): Saya melakukan override pada metode save(). Di dalam metode ini, saya menanamkan logika algoritma untuk menghasilkan booking_number secara otomatis dengan format unik (BK-YYYYMMDD-XXXX). Ini memastikan tidak ada duplikasi nomor antrian tanpa perlu input manual dari pengguna.

iii. Kelas SparePart (Komponen Pendukung)

Kelas ini merepresentasikan penggunaan suku cadang dalam satu sesi servis.

Enkapsulasi Logika: Saya menanamkan logika perhitungan otomatis pada metode save(). Saat mekanik memasukkan quantity dan unit_price, sistem secara otomatis menghitung total_price sebelum data disimpan ke database. Hal ini mengurangi risiko kesalahan hitung (human error).

iv. Kelas Invoice (Entitas Keuangan)

Kelas ini berfungsi sebagai dokumen bukti pembayaran yang sah.

Relasi One-to-One: Menggunakan `OneToOneField` ke `ServiceBooking`, memastikan bahwa satu sesi servis hanya memiliki satu tagihan unik.

Otomatisasi Finansial: Pada metode `save()`, sistem secara otomatis melakukan:

Generate Nomor Invoice: Membuat kode unik INV-YYYYMMDD-XXXX.

Kalkulasi Total: Menjumlahkan biaya jasa (`service_cost`), biaya suku cadang (`spare_parts_cost`), dan biaya tambahan.

Hitung Kembalian: Mengkalkulasi `change_amount` berdasarkan uang yang dibayarkan (`paid_amount`).

v. Kelas `ServiceType` & `AdditionalService`

Kelas-kelas ini berperan sebagai Master Data untuk standarisasi jenis layanan dan harga dasar, memudahkan admin dalam mengelola katalog bengkel secara dinamis.

Kesimpulan Penerapan OOP pada Kode

- **Inheritance (Pewarisan):** Semua kelas mewarisi fitur-fitur canggih dari `models.Model` milik Django, sehingga memudahkan operasi *Create, Read, Update, Delete* (CRUD).
- **Encapsulation (Pembungkusan):** Logika bisnis (seperti pembuatan nomor otomatis dan perhitungan biaya) disembunyikan di dalam metode `save()` masing-masing kelas, sehingga bagian lain dari program hanya perlu memanggil fungsi simpan tanpa mengetahui kompleksitas di baliknya.
- **Association (Asosiasi):** Hubungan antar objek dibangun secara kuat menggunakan `ForeignKey` dan `OneToOneField` untuk merepresentasikan relasi dunia nyata antara Pelanggan, Mekanik, dan Kendaraan.

Nama File: `account/models.py` **Kode Program:**

```

from django.db import models
from django.contrib.auth.models import AbstractUser

# Create your models here.

class User(AbstractUser):
    """
    Custom User Model untuk MotoFix
    Extends Django AbstractUser untuk menambahkan field tambahan
    """
    ROLE_CHOICES = (
        ('customer', 'Pelanggan'),
        ('mechanic', 'Mekanik'),
        ('cashier', 'Kasir'),
        ('admin', 'Admin'),
    )

    SPECIALIZATION_CHOICES = (
        ('umum', 'Umum'),
        ('mesin', 'Mesin'),
        ('elektrikal', 'Elektrikal'),
        ('ban', 'Ban'),
        ('transmisi', 'Transmisi'),
        ('suspensi', 'Suspensi'),
        ('rem', 'Rem'),
        ('karburator', 'Karburator/Injeksi'),
        ('rantai_sproket', 'Rantai dan Sproket'),
        ('kopling', 'Kopling'),
    )

    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='customer')
    phone = models.CharField(max_length=15, blank=True, null=True)
    address = models.TextField(blank=True, null=True)
    specialization = models.CharField(max_length=50, choices=SPECIALIZATION_CHOICES, )
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'users'
        verbose_name = 'User'
        verbose_name_plural = 'Users'

    def __str__(self):

```

```

        return f"{self.username} ({self.get_role_display()})"

@property
def is_customer(self):
    return self.role == 'customer'

@property
def is_mechanic(self):
    return self.role == 'mechanic'

@property
def is_cashier(self):
    return self.role == 'cashier'

@property
def is_admin_user(self):
    return self.role == 'admin'

```

Syntax ini mendefinisikan model `User` kustom yang memperluas (**inheritance**) fitur bawaan Django (`AbstractUser`). Tujuannya adalah untuk menyesuaikan data pengguna dengan kebutuhan spesifik bengkel yang tidak tersedia di standar Django.

Poin-poin kuncinya adalah:

- i. **Modifikasi User Bawaan:** Dengan menggunakan `AbstractUser` , sistem tetap memanfaatkan fitur keamanan login/password bawaan Django, namun kita bisa menambahkan kolom baru seperti nomor telepon (`phone`) dan alamat (`address`).
- ii. **Manajemen Peran (Role):** Field `role` ditambahkan untuk membedakan hak akses pengguna menjadi 4 kategori: **Pelanggan**, **Mekanik**, **Kasir**, dan **Admin**.
- iii. **Spesialisasi Mekanik:** Field `specialization` ditambahkan khusus untuk menyimpan keahlian teknis mekanik (misalnya: spesialis mesin atau kelistrikan).
- iv. **Metode Pengecekan Praktis:** Fungsi dengan dekorator `@property` (seperti `is_mechanic` , `is_admin_user`) dibuat agar sistem dapat dengan mudah memverifikasi peran pengguna saat login atau mengakses halaman tertentu (True/False).

Nama File: `account/views.py` **Kode Program:**

```

from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login as auth_login, logout as auth_logout
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.http import HttpResponseRedirect
from functools import wraps
from .models import User

# Decorator untuk cek role
def customer_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('accounts:login')
        if not request.user.is_customer:
            messages.error(request, 'Akses ditolak! Halaman ini khusus untuk pelanggan')
            return redirect('admin_hub:admin_dashboard')
        return view_func(request, *args, **kwargs)
    return wrapper

def staff_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('accounts:login')
        if request.user.is_customer:
            messages.error(request, 'Akses ditolak! Halaman ini khusus untuk staff.')
            return redirect('customer:customer_dashboard')
        return view_func(request, *args, **kwargs)
    return wrapper

def admin_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('accounts:login')
        if not request.user.is_admin_user:
            messages.error(request, 'Akses ditolak! Halaman ini khusus untuk admin.')
            return redirect('admin_hub:admin_dashboard')
        return view_func(request, *args, **kwargs)
    return wrapper

def cashier_or_admin_required(view_func):

```

```

@wraps(view_func)
def wrapper(request, *args, **kwargs):
    if not request.user.is_authenticated:
        return redirect('accounts:login')
    if not (request.user.is_admin_user or request.user.is_cashier):
        messages.error(request, 'Akses ditolak! Halaman ini khusus untuk kasir atau mekanik')
        return redirect('admin_hub:booking_list')
    return view_func(request, *args, **kwargs)
return wrapper

# Autentikasi & Akun
def login(request):
    """Halaman login untuk semua user"""
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(request, username=username, password=password)

        if user is not None:
            auth_login(request, user)
            messages.success(request, f'Selamat datang, {user.username}!')

            # Redirect berdasarkan role
            if user.is_admin_user or user.is_cashier or user.is_mechanic:
                return redirect('admin_hub:admin_dashboard')
            else:
                return redirect('customer:customer_dashboard')
        else:
            messages.error(request, 'Username atau password salah!')

    return render(request, 'accounts/login.html')

def logout(request):
    """Proses logout"""
    auth_logout(request)
    messages.info(request, 'Anda telah logout.')
    return redirect('accounts:login')

def register(request):
    """Halaman pendaftaran pelanggan baru"""
    if request.method == 'POST':
        username = request.POST.get('username')

```

```

first_name = request.POST.get('first_name', '').strip()
last_name = request.POST.get('last_name', '').strip()
email = request.POST.get('email')
password = request.POST.get('password')
password_confirm = request.POST.get('password_confirm')
phone = request.POST.get('phone')
address = request.POST.get('address')

# Validasi
if password != password_confirm:
    messages.error(request, 'Password tidak sama!')
    return render(request, 'accounts/register.html')

if User.objects.filter(username=username).exists():
    messages.error(request, 'Username sudah digunakan!')
    return render(request, 'accounts/register.html')

if User.objects.filter(email=email).exists():
    messages.error(request, 'Email sudah terdaftar!')
    return render(request, 'accounts/register.html')

# Buat user baru
try:
    user = User.objects.create_user(
        username=username,
        first_name=first_name,
        last_name=last_name,
        email=email,
        password=password,
        role='customer',
        phone=phone,
        address=address
    )
    messages.success(request, 'Registrasi berhasil! Silakan login.')
    return redirect('accounts:login')
except Exception as e:
    messages.error(request, f'Registrasi gagal: {str(e)}')
# On error fall through and re-render with previous form values
form_data = request.POST
return render(request, 'accounts/register.html', {'form_data': form_data})

return render(request, 'accounts/register.html')

```


Syntax ini berfungsi sebagai **Pengendali Akses dan Autentikasi** pengguna dalam sistem.

Logikanya terbagi menjadi tiga bagian utama:

i. **Sistem Keamanan (*Decorators*):**

Bagian atas kode (seperti `customer_required`, `admin_required`) berfungsi sebagai "satpam". Kode ini ditempelkan pada halaman lain untuk mengecek: "Apakah pengguna ini boleh masuk?".

- Jika **Pelanggan** mencoba masuk halaman Admin, sistem akan menolaknya.
- Jika **Belum Login**, sistem akan melemparnya kembali ke halaman Login.

ii. **Logika Login Cerdas:**

Fungsi `login` tidak hanya mengecek password, tapi juga **mengarahkan pengguna secara otomatis**.

- Jika yang login adalah **Admin/Mekanik/Kasir**, mereka langsung diarahkan ke `admin_dashboard`.
- Jika yang login adalah **Pelanggan**, mereka diarahkan ke `customer_dashboard`.

iii. **Registrasi Pelanggan:**

Fungsi `register` menangani pendaftaran pengguna baru. Sistem akan memvalidasi agar *username* dan *email* tidak kembar, memastikan konfirmasi password cocok, dan secara otomatis menetapkan peran (*role*) akun baru tersebut sebagai '**customer**'.

Nama File: `customer/urls.py` **Kode Program:**

```
from django.urls import path
from . import views

app_name = 'customer'

urlpatterns = [
    # Panel Pelanggan (Customer)
    path('dashboard/', views.customer_dashboard, name='customer_dashboard'),
    path('motor/add/', views.motor_add, name='motor_add'),
    path('motor/<int:pk>/edit/', views.motor_edit, name='motor_edit'),
    path('motor/<int:pk>/delete/', views.motor_delete, name='motor_delete'),
    path('booking/', views.booking_create, name='booking_create'),
    path('history/', views.service_history, name='service_history'),
    path('invoice/<int:pk>/', views.invoice_detail, name='invoice_detail'),
]
```

Syntax ini berfungsi sebagai **Peta Navigasi (Router)** khusus untuk halaman-halaman yang diakses oleh pelanggan. Kode ini menghubungkan alamat URL di browser dengan fungsi logika (*views*) yang sesuai.

Poin utamanya:

- i. **Identitas Modul (`app_name`):** Memberi label `'customer'` agar sistem mudah membedakan antara URL milik pelanggan dengan URL milik admin.
- ii. **Manajemen Motor:** Alamat untuk menambah, mengedit, dan menghapus motor. Kode `\<int:pk\>` berfungsi sebagai **penangkap ID unik**, sehingga sistem tahu motor *mana* yang sedang diedit atau dihapus.
- iii. **Alur Servis:** Menyediakan jalan menuju fitur utama seperti Dashboard, Form Booking, Riwayat Servis, dan tampilan Invoice digital.

Nama File: `customer/views.py` **Kode Program:**

```

from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from django.utils import timezone
from accounts.views import customer_required
from .models import Motor, ServiceType, ServiceBooking, Invoice
import random

# Panel Pelanggan (Customer)
@customer_required
def customer_dashboard(request):
    """Dashboard utama & tracking status aktif"""
    # Get customer's motors
    motors = Motor.objects.filter(owner=request.user)

    # Get active bookings (not paid or cancelled)
    active_bookings = ServiceBooking.objects.filter(
        customer=request.user
    ).exclude(
        status__in=['paid', 'cancelled']
    ).select_related('motor', 'service_type', 'mechanic').order_by('-booking_date')[:3]

    # Get recent completed services
    completed_services = ServiceBooking.objects.filter(
        customer=request.user,
        status__in=['paid', 'finished']
    ).select_related('motor', 'service_type').order_by('-finish_date')[:3]

    context = {
        'motors': motors,
        'motors_count': motors.count(),
        'active_bookings': active_bookings,
        'active_bookings_count': active_bookings.count(),
        'completed_services': completed_services,
        'total_services': ServiceBooking.objects.filter(customer=request.user).count()
    }

    return render(request, 'customer/dashboard.html', context)

@customer_required
def motor_add(request):
    """Form tambah motor baru"""
    if request.method == 'POST':
        try:

```

```

# Get form data
license_plate = request.POST.get('license_plate', '').strip().upper()
brand = request.POST.get('brand', '').strip()
model = request.POST.get('model', '').strip()
year = request.POST.get('year', '').strip()

print(request.POST)

# engine_number = request.POST.get('engine_number', '').strip()
# frame_number = request.POST.get('frame_number', '').strip()
# notes = request.POST.get('notes', '').strip()

# Validation
if not all([license_plate, brand, model, year]):
    messages.error(request, 'Plat nomor`, merk, model, tahun, dan warna harus diisi')
    return render(request, 'customer/dashboard.html', {'post_data': request.POST})

# Check if license plate already exists
if Motor.objects.filter(license_plate=license_plate).exists():
    messages.error(request, f'Motor dengan plat nomor {license_plate} sudah ada')
    return render(request, 'customer/motor_add.html', {'post_data': request.POST})

# Validate year
try:
    year_int = int(year)
    if year_int < 1900 or year_int > timezone.now().year + 1:
        messages.error(request, 'Tahun motor tidak valid!')
        return render(request, 'customer/motor_add.html', {'post_data': request.POST})
except ValueError:
    messages.error(request, 'Tahun harus berupa angka!')
    return render(request, 'customer/motor_add.html', {'post_data': request.POST})

print(request.user)

# Create motor
motor = Motor.objects.create(
    owner=request.user,
    license_plate=license_plate,
    brand=brand,
    model=model,
    year=year_int,
)

messages.success(request, f'Motor {motor.license_plate} berhasil ditambahkan')
return redirect('customer:customer_dashboard')

```

```

    except Exception as e:
        messages.error(request, f'Gagal menambahkan motor: {str(e)}')
        return render(request, 'customer/motor_add.html', {'post_data': request.POST})

    return render(request, 'customer/motor_add.html')

@customer_required
def motor_edit(request, pk):
    """Form edit data motor"""
    motor = get_object_or_404(Motor, pk=pk, owner=request.user)

    if request.method == 'POST':
        try:
            # Get form data
            license_plate = request.POST.get('license_plate', '').strip().upper()
            brand = request.POST.get('brand', '').strip()
            model = request.POST.get('model', '').strip()
            year = request.POST.get('year', '').strip()

            # Validation
            if not all([license_plate, brand, model, year]):
                messages.error(request, 'Plat nomor, merk, model, tahun, dan warna harus diisi')
                return render(request, 'customer/motor_edit.html', {'motor': motor})

            # Check if license plate already exists (except current motor)
            if Motor.objects.filter(license_plate=license_plate).exclude(pk=motor.pk):
                messages.error(request, f'Motor dengan plat nomor {license_plate} sudah ada')
                return render(request, 'customer/motor_edit.html', {'motor': motor})

            # Validate year
            try:
                year_int = int(year)
                if year_int < 1900 or year_int > timezone.now().year + 1:
                    messages.error(request, 'Tahun motor tidak valid!')
                    return render(request, 'customer/motor_edit.html', {'motor': motor})
            except ValueError:
                messages.error(request, 'Tahun harus berupa angka!')
                return render(request, 'customer/motor_edit.html', {'motor': motor})

            # Update motor
            motor.license_plate = license_plate
            motor.brand = brand
            motor.model = model

```

```

        motor.year = year_int

        motor.save()

        messages.success(request, f'Data motor {motor.license_plate} berhasil diu
        return redirect('customer:customer_dashboard')

    except Exception as e:
        messages.error(request, f'Gagal mengupdate motor: {str(e)}')

    return render(request, 'customer/motor_edit.html', {'motor': motor})

@customer_required
def motor_delete(request, pk):
    """Hapus motor"""
    motor = get_object_or_404(Motor, pk=pk, owner=request.user)

    if request.method == 'POST':
        try:
            motor.delete()
            messages.success(request, f'Motor {motor.license_plate} berhasil dihapus!
            return redirect('customer:customer_dashboard')
        except Exception as e:
            messages.error(request, f'Gagal menghapus motor: {str(e)}')
            return redirect('customer:customer_dashboard')

    return render(request, 'customer/motor_delete_confirm.html', {'motor': motor})

@customer_required
def booking_create(request):
    """Form buat antrian (Booking/Walk-in)"""
    # Get customer's motors
    motors = Motor.objects.filter(owner=request.user)

    # Get active service types
    service_types = ServiceType.objects.filter(is_active=True)

    if request.method == 'POST':
        try:
            # Get form data
            print(request.POST)
            motor_id = request.POST.get('motor')
            first_name = request.POST.get('first_name', '').strip()

```

```

service_type_id = request.POST.get('service')
complaint = request.POST.get('complaint', '').strip()
booking_type = request.POST.get('booking_type', 'booking')

# Validation
if not all([motor_id, service_type_id, complaint, booking_type]):
    messages.error(request, 'Motor, jenis servis, dan keluhan harus diisi')
    return render(request, 'customer/booking_create.html', {
        'motors': motors,
        'service_types': service_types,
        'post_data': request.POST,
        'booking_type': booking_type,
    })

# Get motor and service type
motor = get_object_or_404(Motor, pk=motor_id, owner=request.user)
service_type = get_object_or_404(ServiceType, pk=service_type_id, is_active=True)

# Generate booking number (format: BK-YYYYMMDD-XXXX)
today = timezone.now()
date_str = today.strftime('%Y%m%d')
random_num = random.randint(1000, 9999)
booking_number = f'BK-{date_str}-{random_num}'

# Make sure booking number is unique
while ServiceBooking.objects.filter(booking_number=booking_number).exists:
    random_num = random.randint(1000, 9999)
    booking_number = f'BK-{date_str}-{random_num}'

while ServiceBooking.objects.filter(motor=motor, status__in=['pending', 'confirmed']).exists:
    messages.error(request, f'Motor {motor.license_plate} sudah memiliki booking')
    return render(request, 'customer/booking_create.html', {
        'motors': motors,
        'service_types': service_types,
        'post_data': request.POST,
        'booking_type': booking_type,
    })

# Create booking
# update user's name/phone if provided
if first_name:
    request.user.first_name = first_name

```

```

        request.user.save()

    booking = ServiceBooking.objects.create(
        booking_number=booking_number,
        motor=motor,
        customer=request.user,
        service_type=service_type,
        booking_type=booking_type,
        status='pending',
        complaint=complaint,
        booking_date=today,
    )

    messages.success(request, f'Booking berhasil! Nomor booking: {booking.booking_number}')
    return redirect('customer:customer_dashboard')

except Exception as e:
    messages.error(request, f'Gagal membuat booking: {str(e)}')

context = {
    'motors': motors,
    'service_types': service_types,
}

return render(request, 'customer/booking_create.html', context)

@customer_required
def service_history(request):
    """Daftar riwayat servis selesai"""
    # Get all bookings for this customer
    bookings = ServiceBooking.objects.filter(
        customer=request.user
    ).select_related('motor', 'service_type', 'mechanic').order_by('-booking_date')

    # Filter by status if provided
    status_filter = request.GET.get('status', '')
    if status_filter:
        bookings = bookings.filter(status=status_filter)

    context = {
        'bookings': bookings,
        'status_filter': status_filter,
        'status_choices': ServiceBooking.STATUS_CHOICES,
    }

```



```

    }

    return render(request, 'customer/service_history.html', context)

@customer_required
def invoice_detail(request, pk):
    """Detail biaya dan nota servis"""
    booking = get_object_or_404(ServiceBooking, pk=pk, customer=request.user)

    # Try to get invoice
    try:
        invoice = Invoice.objects.get(service_booking=booking)
    except Invoice.DoesNotExist:
        invoice = None

    context = {
        'booking': booking,
        'invoice': invoice,
    }

    return render(request, 'customer/invoice_detail.html', context)

```

1. customer_dashboard (Halaman Utama)

Ini adalah pusat informasi bagi pelanggan setelah login.

- **Fungsi:** Mengambil dan menampilkan ringkasan data.
- **Data yang diambil:**
 - Daftar motor milik user.
 - **Booking Aktif:** Servis yang statusnya belum selesai/lunas (untuk dipantau statusnya).
 - **Riwayat Terakhir:** 3 servis terakhir yang sudah selesai.
 - Statistik jumlah motor dan total servis.

2. motor_add (Tambah Motor)

- **Fungsi:** Menangani formulir pendaftaran motor baru.
- **Validasi:**
 - Mengecek apakah semua kolom (Plat nomor, Merk, Model, Tahun) terisi.
 - **Cek Duplikasi:** Memastikan plat nomor belum pernah terdaftar di sistem.

- **Validasi Tahun:** Memastikan input tahun berupa angka dan masuk akal (antara 1900 sampai tahun sekarang).
- **Proses:** Jika valid, data disimpan ke database dan dihubungkan dengan akun user yang sedang login.

3. `motor_edit` (Edit Motor)

- **Fungsi:** Mengubah data motor yang sudah ada.
- **Keamanan:** Menggunakan `get_object_or_404(..., owner=request.user)` untuk memastikan user hanya bisa mengedit motor miliknya sendiri (bukan milik orang lain).
- **Validasi:** Mirip dengan tambah motor, namun pengecekan duplikasi plat nomor dikecualikan untuk motor yang sedang diedit itu sendiri.

4. `motor_delete` (Hapus Motor)

- **Fungsi:** Menghapus data motor.
- **Proses:** Menampilkan halaman konfirmasi hapus. Jika user setuju (klik tombol hapus/POST), data dihapus dari database.

5. `booking_create` (Buat Booking Servis)

Ini adalah logika inti untuk transaksi reservasi.

- **Input:** User memilih Motor, Jenis Servis, dan menulis Keluhan.
- **Logika Penting:**
 - **Generate Nomor Booking:** Membuat kode unik (contoh: `BK-20240101-1234`) secara acak dan memastikan tidak ada yang kembar.
 - **Cek Booking Ganda:** Mencegah user melakukan booking lagi untuk motor yang status servisnya masih berjalan (*pending* atau *in_progress*).
- **Output:** Jika sukses, data tersimpan dengan status awal '**pending**' (Menunggu).

6. `service_history` (Riwayat Servis)

- **Fungsi:** Menampilkan daftar panjang semua servis yang pernah dilakukan.
- **Fitur:** Mendukung **Filter Status**. User bisa memilih untuk hanya melihat yang "Selesai", "Dibatalkan", atau "Menunggu" melalui parameter URL (contoh: `?status=finished`).

7. `invoice_detail` (Lihat Nota)

- **Fungsi:** Menampilkan detail tagihan untuk satu transaksi servis tertentu.

- **Proses:** Mengambil data Booking berdasarkan ID. Kemudian mencoba mencari data **Invoice** pasangannya.
 - Jika invoice belum dibuat oleh kasir (misal servis baru mulai), maka `invoice` akan bernilai `None` (kosong).
 - Jika sudah ada, detail biaya akan ditampilkan.

Nama File: `admin_hub/urls.py` **Kode Program:**

```

from django.urls import path
from . import views

app_name = 'admin_hub'

urlpatterns = [
    # Dashboard
    path('dashboard/', views.admin_dashboard, name='admin_dashboard'),

    # Manajemen Tugas
    path('queue/<int:pk>/allocate/', views.allocate_mechanic, name='allocate_mechanic'),
    path('service/<int:pk>/start/', views.service_start, name='service_start'),
    path('service/<int:pk>/update/', views.service_update, name='service_update'),
    path('service/<int:pk>/finish/', views.service_finish, name='service_finish'),

    # Kasir & Pembayaran
    path('payment/<int:pk>/', views.process_payment, name='process_payment'),
    path('invoice/create/<int:pk>/', views.create_invoice, name='create_invoice'),
    path('invoice/<int:pk>/', views.invoice_detail, name='invoice_detail'),

    # Master Data
    path('services/', views.service_list, name='service_list'),
    path('services/add/', views.service_add, name='service_add'),
    path('services/<int:pk>/edit/', views.service_edit, name='service_edit'),
    path('services/<int:pk>/delete/', views.service_delete, name='service_delete'),
    path('mechanics/', views.mechanic_list, name='mechanic_list'),

    # Manajemen Booking Service
    path('bookings/', [views.booking](http://views.booking)_list, name='booking_list'),
    path('bookings/create/', [views.booking](http://views.booking)_create, name='booking_create'),
    path('bookings/<int:pk>/', [views.booking](http://views.booking)_detail, name='booking_detail'),
    path('bookings/<int:pk>/approve/', [views.booking](http://views.booking)_approve, name='booking_approve'),
    path('bookings/<int:pk>/assign-mechanic/', [views.booking](http://views.booking)_assign_mechanic, name='booking_assign_mechanic'),
    path('bookings/<int:pk>/start/', [views.booking](http://views.booking)_start, name='booking_start'),
    path('bookings/<int:pk>/finish/', [views.booking](http://views.booking)_finish, name='booking_finish'),
    path('bookings/<int:pk>/cancel/', [views.booking](http://views.booking)_cancel, name='booking_cancel'),
    path('bookings/<int:pk>/update-status/', [views.booking](http://views.booking)_update_status, name='booking_update_status'),

    # Manajemen User (CRUD)
    path('users/', views.user_list, name='user_list'),
    path('users/create/', views.user_create, name='user_create'),
    path('users/<int:pk>/', views.user_detail, name='user_detail'),
    path('users/<int:pk>/update/', views.user_update, name='user_update'),

```

```
path('users/<int:pk>/delete/', views.user_delete, name='user_delete'),  
]
```

Penjelasan Kode Program:

Syntax ini berfungsi sebagai **Router** utama untuk panel Administrator. Kode ini mengatur bagaimana sistem merespons alamat URL yang diakses oleh admin dan menghubungkannya ke fungsi logika (*views*) yang sesuai.

Struktur *routing* ini dikelompokkan menjadi beberapa kategori fungsional untuk memudahkan pengelolaan:

- i. **Manajemen Operasional (Tugas & Booking):** URL untuk menangani alur kerja servis mulai dari persetujuan *booking*, penunjukan mekanik (*allocate/assign*), memulai pengerjaan (*start*), hingga menandai servis selesai (*finish*).
- ii. **Kasir & Keuangan:** URL yang menangani proses pembayaran dan pembuatan *invoice* digital setelah servis selesai.
- iii. **Master Data:** URL untuk mengelola data referensi sistem, seperti menambah/mengedit jenis layanan (*services*) dan daftar mekanik.
- iv. **Manajemen Pengguna (User):** URL untuk fitur CRUD (*Create, Read, Update, Delete*) akun pengguna, memungkinkan admin menambah atau menghapus akun pelanggan/staf.
- v. **Parameter Dinamis (<int:pk>):** Kode ini digunakan untuk menangkap **ID unik** dari data yang sedang diproses, sehingga sistem tahu data mana (misalnya: booking ID 15) yang harus diedit, disetujui, atau dihapus.

Nama File: admin_hub/views.py **Kode Program:**

```

from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.utils import timezone
from accounts.models import User
from accounts.views import staff_required, admin_required, cashier_or_admin_required
from customer.models import ServiceBooking, Motor, ServiceType

# Panel Admin & Manajemen (Integrated Hub)
@staff_required
def admin_dashboard(request):
    """Monitoring seluruh antrian & status bengkel"""
    # Statistik booking
    total_bookings = ServiceBooking.objects.count()
    pending_bookings = ServiceBooking.objects.filter(status='pending').count()
    in_progress_bookings = ServiceBooking.objects.filter(status='in_progress').count()
    finished_bookings = ServiceBooking.objects.filter(status='finished').count()

    context = {
        'total_bookings': total_bookings,
        'pending_bookings': pending_bookings,
        'in_progress_bookings': in_progress_bookings,
        'finished_bookings': finished_bookings,
    }

    return render(request, 'admin_hub/dashboard.html', context)

# === Manajemen Tugas ===
@admin_required
def allocate_mechanic(request, pk):
    """Proses admin menugaskan mekanik"""
    return render(request, 'admin_hub/allocate_mechanic.html', {'pk': pk})

@admin_required
def service_start(request, pk):
    """Trigger mekanik mulai bekerja (via Admin)"""
    return render(request, 'admin_hub/service_start.html', {'pk': pk})

@admin_required
def service_update(request, pk):
    """Input suku cadang & jasa tambahan (Modal)"""
    return render(request, 'admin_hub/service_update.html', {'pk': pk})

@admin_required

```

```

def service_finish(request, pk):
    """Menandai pengerjaan mekanik selesai"""
    return render(request, 'admin_hub/service_finish.html', {'pk': pk})

# === Kasir & Pembayaran ===
@admin_required
def process_payment(request, pk):
    """Form kasir untuk konfirmasi pembayaran"""
    return render(request, 'admin_hub/process_payment.html', {'pk': pk})

# === Master Data ===
@admin_required
def service_list(request):
    """List semua jenis layanan (Master Data)"""
    from customer.models import ServiceType
    services = ServiceType.objects.all().order_by('name')
    context = {
        'services': services,
    }
    return render(request, 'admin_hub/service_list.html', context)

@admin_required
def service_add(request):
    """Form tambah jenis layanan baru"""
    from customer.models import ServiceType

    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description', '')
        base_price = request.POST.get('base_price')
        estimated_duration = request.POST.get('estimated_duration')

        # Validasi
        if not name or not base_price or not estimated_duration:
            messages.error(request, 'Semua field harus diisi!')
            return render(request, 'admin_hub/service_add.html', {'form_data': request.POST})

        try:
            service = ServiceType.objects.create(
                name=name,
                description=description,
                base_price=float(base_price),
                estimated_duration=int(estimated_duration),
            )

```

```

        is_active=True
    )
    messages.success(request, f'Layanan "{name}" berhasil ditambahkan!')
    return redirect('admin_hub:service_list')
except Exception as e:
    messages.error(request, f'Gagal menambahkan layanan: {str(e)}')
    return render(request, 'admin_hub/service_add.html', {
        'form_data': request.POST
    })

return render(request, 'admin_hub/service_add.html')

@admin_required
def service_edit(request, pk):
    """Edit jenis layanan"""
    from customer.models import ServiceType
    service = get_object_or_404(ServiceType, pk=pk)

    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description', '')
        base_price = request.POST.get('base_price')
        estimated_duration = request.POST.get('estimated_duration')
        is_active = request.POST.get('is_active') == 'on'

        # Validasi
        if not name or not base_price or not estimated_duration:
            messages.error(request, 'Semua field harus diisi!')
            return render(request, 'admin_hub/service_edit.html', {
                'service': service,
            })

        try:
            service.name = name
            service.description = description
            service.base_price = float(base_price)
            service.estimated_duration = int(estimated_duration)
            service.is_active = is_active
            service.save()
            messages.success(request, f'Layanan "{name}" berhasil diupdate!')
            return redirect('admin_hub:service_list')
        except Exception as e:
            messages.error(request, f'Gagal mengupdate layanan: {str(e)}')

```



```

        return render(request, 'admin_hub/service_edit.html', {
            'service': service,
        })

    return render(request, 'admin_hub/service_edit.html', {
        'service': service,
    })

@admin_required
def service_delete(request, pk):
    """Hapus jenis layanan"""
    from customer.models import ServiceType
    service = get_object_or_404(ServiceType, pk=pk)

    if request.method == 'POST':
        try:
            # Cek apakah layanan sudah digunakan dalam booking
            if ServiceBooking.objects.filter(service_type=service).exists():
                messages.error(request,
                    f'Layanan "{service.name}" tidak dapat dihapus karena sudah digunak
                return redirect('admin_hub:service_list')

            service_name = service.name
            service.delete()
            messages.success(request, f'Layanan "{service_name}" berhasil dihapus!')
        except Exception as e:
            messages.error(request, f'Gagal menghapus layanan: {str(e)}')

    return redirect('admin_hub:service_list')

return redirect('admin_hub:service_list')

@admin_required
def mechanic_list(request):
    """List data mekanik"""
    mechanics = User.objects.filter(role='mechanic')
    return render(request, 'admin_hub/mechanic_list.html', {'mechanics': mechanics})

# === Manajemen User (CRUD) ===
@admin_required
def user_list(request):
    """List semua user"""
    users = User.objects.all().order_by('-created_at')

```

```

        return render(request, 'admin_hub/user_list.html', {'users': users})

@admin_required
def user_create(request):
    """Form tambah user baru"""
    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        password_confirm = request.POST.get('password_confirm')
        role = request.POST.get('role')
        phone = request.POST.get('phone')
        address = request.POST.get('address')
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')
        specialization = request.POST.get('specialization')

        # Validasi
        if password != password_confirm:
            messages.error(request, 'Password tidak sama!')
            return render(request, 'admin_hub/user_form.html', {
                'form_data': request.POST,
                'action': 'create'
            })

        if User.objects.filter(username=username).exists():
            messages.error(request, 'Username sudah digunakan!')
            return render(request, 'admin_hub/user_form.html', {
                'form_data': request.POST,
                'action': 'create'
            })

        if email and User.objects.filter(email=email).exists():
            messages.error(request, 'Email sudah terdaftar!')
            return render(request, 'admin_hub/user_form.html', {
                'form_data': request.POST,
                'action': 'create'
            })

        # Buat user baru
        try:
            user = User.objects.create_user(
                username=username,

```

```

        email=email,
        password=password,
        role=role,
        phone=phone,
        address=address,
        first_name=first_name,
        last_name=last_name,
        specialization=specialization
    )
    messages.success(request,
        f'User {username} berhasil ditambahkan! '
        f'Username: {username} | Password: {password} '
        f'(Berikan informasi login ini kepada user)')
    return redirect('admin_hub:user_list')
except Exception as e:
    messages.error(request, f'Gagal menambahkan user: {str(e)}')

return render(request, 'admin_hub/user_form.html', {'action': 'create'})

```

@admin_required

```

def user_detail(request, pk):
    """Detail user"""
    user = get_object_or_404(User, pk=pk)
    return render(request, 'admin_hub/user_detail.html', {'user_obj': user})

```

@admin_required

```

def user_update(request, pk):
    """Form edit user"""
    user = get_object_or_404(User, pk=pk)

    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        role = request.POST.get('role')
        phone = request.POST.get('phone')
        address = request.POST.get('address')
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')
        specialization = request.POST.get('specialization')
        is_active = request.POST.get('is_active') == 'on'

        # Validasi username unique (kecuali user sendiri)
        if username != user.username and User.objects.filter(username=username).exists:

```

```

        messages.error(request, 'Username sudah digunakan!')
        return render(request, 'admin_hub/user_form.html', {
            'user_obj': user,
            'action': 'update'
        })

# Validasi email unique (kecuali user sendiri)
if email and email != user.email and User.objects.filter(email=email).exists():
    messages.error(request, 'Email sudah terdaftar!')
    return render(request, 'admin_hub/user_form.html', {
        'user_obj': user,
        'action': 'update'
    })

# Update user
try:
    user.username = username
    user.email = email
    user.role = role
    user.phone = phone
    user.address = address
    user.first_name = first_name
    user.last_name = last_name
    user.specialization = specialization
    user.is_active = is_active

    # Update password jika diisi
    new_password = request.POST.get('password')
    if new_password:
        password_confirm = request.POST.get('password_confirm')
        if new_password != password_confirm:
            messages.error(request, 'Password tidak sama!')
            return render(request, 'admin_hub/user_form.html', {
                'user_obj': user,
                'action': 'update'
            })
        user.set_password(new_password)

    user.save()
    messages.success(request, f'User {username} berhasil diupdate!')
    return redirect('admin_hub:user_detail', pk=user.pk)
except Exception as e:
    messages.error(request, f'Gagal mengupdate user: {str(e)}')

```

```

        return render(request, 'admin_hub/user_form.html', {
            'user_obj': user,
            'action': 'update'
        })

@admin_required
def user_delete(request, pk):
    """Hapus user"""
    user = get_object_or_404(User, pk=pk)

    if request.method == 'POST':
        username = user.username
        try:
            user.delete()
            messages.success(request, f'User {username} berhasil dihapus!')
            return redirect('admin_hub:user_list')
        except Exception as e:
            messages.error(request, f'Gagal menghapus user: {str(e)}')
            return redirect('admin_hub:user_detail', pk=pk)

    return render(request, 'admin_hub/user_delete.html', {'user_obj': user})

# === Manajemen Booking Service ===
@admin_required
def booking_list(request):
    """List semua booking service dengan filter status"""
    status = request.GET.get('status', '')

    bookings = ServiceBooking.objects.select_related(
        'customer', 'motor', 'service_type', 'mechanic'
    ).order_by('-booking_date')

    if status:
        bookings = bookings.filter(status=status)

    context = {
        'bookings': bookings,
        'status_choices': ServiceBooking.STATUS_CHOICES,
        'current_status': status,
        'total_bookings': ServiceBooking.objects.count(),
        'pending_count': ServiceBooking.objects.filter(status='pending').count(),
        'assigned_count': ServiceBooking.objects.filter(status='assigned').count(),
    }

```

```

        'in_progress_count': ServiceBooking.objects.filter(status='in_progress').count(),
        'finished_count': ServiceBooking.objects.filter(status='finished').count(),
        'paid_count': ServiceBooking.objects.filter(status='paid').count(),
    }

    return render(request, 'admin_hub/booking_list.html', context)

@staff_required
def booking_detail(request, pk):
    """Detail booking service"""
    from customer.models import Invoice

    booking = get_object_or_404(ServiceBooking.objects.select_related(
        'customer', 'motor', 'service_type', 'mechanic'
    ).prefetch_related('spare_parts', 'additional_services'), pk=pk)

    # include role case-insensitive to avoid missing mechanics due to unexpected role
    mechanics = User.objects.filter(role__iexact='mechanic').order_by('first_name', 'last_name')

    # Cek apakah invoice sudah ada
    try:
        invoice = Invoice.objects.get(service_booking=booking)
    except Invoice.DoesNotExist:
        invoice = None

    context = {
        'booking': booking,
        'mechanics': mechanics,
        'status_choices': ServiceBooking.STATUS_CHOICES,
        'invoice': invoice,
    }

    return render(request, 'admin_hub/booking_detail.html', context)

@cashier_or_admin_required
def booking_approve(request, pk):
    """Approve booking (ubah status dari pending ke assigned)"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status != 'pending':
        messages.error(request, 'Booking hanya bisa di-approve jika status Menunggu!')
        return redirect('admin_hub:booking_detail', pk=pk)

```

```

try:
    booking.status = 'assigned'
    booking.assigned_date = timezone.now()
    booking.save()
    messages.success(request, f'Booking {booking.booking_number} berhasil di-approve')
    return redirect('admin_hub:booking_detail', pk=pk)
except Exception as e:
    messages.error(request, f'Gagal approve booking: {str(e)}')
    return redirect('admin_hub:booking_detail', pk=pk)

@cashier_or_admin_required
def booking_assign_mechanic(request, pk):
    """Assign mekanik ke booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if request.method == 'POST':
        mechanic_id = request.POST.get('mechanic_id')

        if not mechanic_id:
            messages.error(request, 'Pilih mekanik terlebih dahulu!')
            return redirect('admin_hub:booking_detail', pk=pk)

        try:
            # Try to get the user by id first. Avoid failing if role string differs.
            mechanic = User.objects.get(id=mechanic_id)
            if not mechanic.is_mechanic:
                messages.error(request, 'User yang dipilih bukan mekanik. Periksa peran!')
                return redirect('admin_hub:booking_detail', pk=pk)
            booking.mechanic = mechanic
            booking.status = 'assigned'
            booking.assigned_date = timezone.now()
            booking.save()
            messages.success(request, f'Mekanik {mechanic.get_full_name()} berhasil di-assign')
            return redirect('admin_hub:booking_detail', pk=pk)
        except User.DoesNotExist:
            messages.error(request, 'Mekanik tidak ditemukan!')
            return redirect('admin_hub:booking_detail', pk=pk)
        except Exception as e:
            messages.error(request, f'Gagal assign mekanik: {str(e)}')
            return redirect('admin_hub:booking_detail', pk=pk)

    return redirect('admin_hub:booking_detail', pk=pk)

```

```

@cashier_or_admin_required
def booking_start(request, pk):
    """Mulai pengerjaan booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status != 'assigned':
        messages.error(request, 'Booking hanya bisa dimulai jika sudah ditugaskan!')
        return redirect('admin_hub:booking_detail', pk=pk)

    try:
        booking.status = 'in_progress'
        booking.start_date = timezone.now()
        booking.save()
        messages.success(request, f'Booking {booking.booking_number} mulai dikerjakan')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Exception as e:
        messages.error(request, f'Gagal memulai booking: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)

@cashier_or_admin_required
def booking_finish(request, pk):
    """Selesaikan booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status != 'in_progress':
        messages.error(request, 'Booking hanya bisa diselesaikan jika sedang dikerjakan')
        return redirect('admin_hub:booking_detail', pk=pk)

    try:
        booking.status = 'finished'
        booking.finish_date = timezone.now()
        booking.save()
        messages.success(request, f'Booking {booking.booking_number} telah selesai di')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Exception as e:
        messages.error(request, f'Gagal menyelesaikan booking: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)

@cashier_or_admin_required
def invoice_detail(request, pk):
    """Detail invoice untuk admin/kasir"""
    from customer.models import Invoice, SparePart, AdditionalService

```



```

booking = get_object_or_404(ServiceBooking, pk=pk)

# Get invoice
try:
    invoice = Invoice.objects.get(service_booking=booking)
except Invoice.DoesNotExist:
    messages.error(request, 'Invoice belum dibuat untuk booking ini!')
    return redirect('admin_hub:booking_detail', pk=pk)

# Get spare parts and additional services
spare_parts = SparePart.objects.filter(service_booking=booking)
additional_services = AdditionalService.objects.filter(service_booking=booking)

context = {
    'booking': booking,
    'invoice': invoice,
    'spare_parts': spare_parts,
    'additional_services': additional_services,
}

return render(request, 'admin_hub/invoice_detail.html', context)

@cashier_or_admin_required
def create_invoice(request, pk):
    """Buat invoice untuk booking yang sudah selesai"""
    from customer.models import Invoice, SparePart, AdditionalService
    from decimal import Decimal

    booking = get_object_or_404(ServiceBooking, pk=pk)

    # Cek apakah booking sudah selesai atau paid
    if booking.status not in ['finished', 'paid']:
        messages.error(request, 'Invoice hanya bisa dibuat untuk booking yang sudah selesai')
        return redirect('admin_hub:booking_detail', pk=pk)

    # Cek apakah invoice sudah ada
    try:
        existing_invoice = Invoice.objects.get(service_booking=booking)
        messages.warning(request, 'Invoice sudah dibuat sebelumnya!')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Invoice.DoesNotExist:
        pass

```

```

if request.method == 'POST':
    payment_method = request.POST.get('payment_method', 'cash')
    paid_amount_str = request.POST.get('paid_amount', '0')

    try:
        # Hitung biaya layanan dasar
        service_cost = Decimal(str(booking.service_type.base_price))

        # Hitung total biaya suku cadang
        spare_parts = SparePart.objects.filter(service_booking=booking)
        spare_parts_cost = sum(Decimal(str(sp.price * sp.quantity)) for sp in spare_parts)

        # Hitung total biaya jasa tambahan
        additional_services = AdditionalService.objects.filter(service_booking=booking)
        additional_cost = sum(Decimal(str(ads.price)) for ads in additional_services)

        # Total biaya
        total_cost = service_cost + spare_parts_cost + additional_cost

        # Parse paid amount dari form
        paid_amount = Decimal(str(paid_amount_str))

        # Validasi pembayaran
        if paid_amount < total_cost:
            messages.error(request, f'Uang yang dibayarkan kurang! Total: Rp {total_cost}')
            return redirect('admin_hub:booking_detail', pk=pk)

        # Hitung kembalian
        change_amount = paid_amount - total_cost

        # Buat invoice
        invoice = Invoice.objects.create(
            service_booking=booking,
            invoice_number=f"INV-{booking.booking_number}",
            service_cost=service_cost,
            spare_parts_cost=spare_parts_cost,
            additional_cost=additional_cost,
            total_cost=total_cost,
            payment_method=payment_method,
            paid_amount=paid_amount,
            change_amount=change_amount,
            cashier=request.user
        )

```

```

# Update status booking menjadi paid jika belum
if booking.status == 'finished':
    booking.status = 'paid'
    booking.save()

messages.success(request,
    f'Invoice {invoice.invoice_number} berhasil dibuat! '
    f'Total: Rp {total_cost:,.0f} | Dibayar: Rp {paid_amount:,.0f} | Kembali
return redirect('admin_hub:booking_detail', pk=pk)

except Exception as e:
    messages.error(request, f'Gagal membuat invoice: {str(e)}')
    return redirect('admin_hub:booking_detail', pk=pk)

return redirect('admin_hub:booking_detail', pk=pk)

@cashier_or_admin_required
def booking_cancel(request, pk):
    """Batalan booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status in ['finished', 'paid', 'cancelled']:
        messages.error(request, f'Booking dengan status {booking.get_status_display()}
        return redirect('admin_hub:booking_detail', pk=pk)

    if request.method == 'POST':
        try:
            booking.status = 'cancelled'
            booking.save()
            messages.success(request, f'Booking {booking.booking_number} berhasil dib
            return redirect('admin_hub:booking_list')
        except Exception as e:
            messages.error(request, f'Gagal membatalkan booking: {str(e)}')
            return redirect('admin_hub:booking_detail', pk=pk)

    return render(request, 'admin_hub/booking_cancel.html', {'booking': booking})

@cashier_or_admin_required
def booking_create(request):
    """Buat booking baru (walk-in)"""
    motors = Motor.objects.select_related('owner').all()
    service_types = ServiceType.objects.filter(is_active=True)

```

```

from accounts.models import User
customers = User.objects.filter(role='customer')

if request.method == 'POST':
    motor_id = request.POST.get('motor_id')
    customer_id = request.POST.get('customer_id')
    guest_name = request.POST.get('guest_name', '').strip()
    guest_phone = request.POST.get('guest_phone', '').strip()

    # Walk-in motor fields
    motor_license_plate = request.POST.get('motor_license_plate', '').strip()
    motor_brand = request.POST.get('motor_brand', '').strip()
    motor_model = request.POST.get('motor_model', '').strip()
    motor_year = request.POST.get('motor_year', '').strip()

    service_type_id = request.POST.get('service_type_id')
    complaint = request.POST.get('complaint')
    notes = request.POST.get('notes', '')

    # Validasi: harus ada service_type dan complaint
    if not service_type_id or not complaint:
        messages.error(request, 'Mohon isi jenis layanan dan keluhan pelanggan!')
        return render(request, 'admin_hub/booking_create.html', {
            'motors': motors,
            'service_types': service_types,
            'customers': customers,
            'form_data': request.POST,
        })

    try:
        # === 1. Tentukan Customer ===
        if customer_id:
            # Pelanggan terdaftar
            customer = User.objects.get(id=customer_id)
            guest_password = None
        else:
            # Pelanggan walk-in: buat user guest
            if not guest_name or not guest_phone:
                raise ValueError('Untuk pelanggan walk-in, mohon isi nama dan nomor telepon')

            # Generate unique username yang lebih pendek
            import time, random, string
            # Ambil 4 digit terakhir nomor telepon

```

```

phone_digits = ''.join(ch for ch in guest_phone if ch.isdigit())
last_digits = phone_digits[-4:] if len(phone_digits) >= 4 else phone_
# Generate 3 digit random
random_suffix = ''.join(random.choices(string.digits, k=3))
uname = f"guest{last_digits}{random_suffix}"

# Pastikan unique
counter = 1
base_uname = uname
while User.objects.filter(username=uname).exists():
    uname = f"{base_uname}{counter}"
    counter += 1

# Generate random password (8 karakter)
guest_password = ''.join(random.choices(string.ascii_letters + string

customer = User.objects.create_user(
    username=uname,
    password=guest_password, # Set password agar bisa login
    role='customer',
    first_name=guest_name,
    phone=guest_phone,
)
customer.save()

# === 2. Tentukan Motor ===
if motor_id:
    # Motor dari database
    motor = Motor.objects.get(id=motor_id)
else:
    # Motor walk-in: input manual
    if not motor_license_plate or not motor_brand or not motor_model:
        raise ValueError('Untuk motor walk-in, mohon isi plat nomor, merk,

    if not motor_year:
        motor_year = timezone.now().year

# Cek apakah motor dengan plat nomor ini sudah ada
motor, created = Motor.objects.get_or_create(
    license_plate=motor_license_plate.upper(),
    defaults={
        'owner': customer,
        'brand': motor_brand,

```

```

        'model': motor_model,
        'year': int(motor_year) if motor_year else timezone.now().year
    }
)

if not created:
    # Motor sudah ada, update owner jika berbeda
    if motor.owner != customer:
        motor.owner = customer
        motor.save()

# === 3. Buat Booking ===
service_type = ServiceType.objects.get(id=service_type_id)

booking = ServiceBooking.objects.create(
    motor=motor,
    customer=customer,
    service_type=service_type,
    complaint=complaint,
    notes=notes,
    booking_type='walk_in',
    status='pending'
)

# Success message dengan info login untuk guest user
if guest_password:
    messages.success(request,
        f'Booking {booking.booking_number} berhasil dibuat! '
        f'Akun pelanggan dibuat: Username: {customer.username} | Password '
        f'(Berikan informasi login ini kepada pelanggan)')
else:
    messages.success(request, f'Booking {booking.booking_number} berhasil

return redirect('admin_hub:booking_detail', pk=booking.pk)

except ValueError as e:
    messages.error(request, str(e))
    return render(request, 'admin_hub/booking_create.html', {
        'motors': motors,
        'service_types': service_types,
        'customers': customers,
        'form_data': request.POST,
    })

```

```

except Exception as e:
    messages.error(request, f'Gagal membuat booking: {str(e)}')
    return render(request, 'admin_hub/booking_create.html', {
        'motors': motors,
        'service_types': service_types,
        'customers': customers,
        'form_data': request.POST,
    })

context = {
    'motors': motors,
    'service_types': service_types,
    'customers': customers,
}

return render(request, 'admin_hub/booking_create.html', context)

@cashier_or_admin_required
def booking_update_status(request, pk):
    """Update status booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if request.method == 'POST':
        new_status = request.POST.get('status')

        # Validasi status transition
        valid_transitions = {
            'pending': ['assigned', 'cancelled'],
            'assigned': ['in_progress', 'cancelled'],
            'in_progress': ['finished', 'cancelled'],
            'finished': ['paid'],
            'paid': [],
            'cancelled': [],
        }

        if new_status not in valid_transitions.get(booking.status, []):
            messages.error(request, f'Transisi status dari {booking.get_status_display()} ke {new_status} tidak valid')
            return redirect('admin_hub:booking_detail', pk=pk)

        try:
            booking.status = new_status

            # Update timestamp sesuai status

```

```

        if new_status == 'assigned' and not booking.assigned_date:
            booking.assigned_date = timezone.now()
        elif new_status == 'in_progress' and not booking.start_date:
            booking.start_date = timezone.now()
        elif new_status == 'finished' and not booking.finish_date:
            booking.finish_date = timezone.now()
        elif new_status == 'paid' and not booking.payment_date:
            booking.payment_date = timezone.now()

        booking.save()
        messages.success(request, f'Status booking berhasil diubah menjadi {booking.new_status}')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Exception as e:
        messages.error(request, f'Gagal update status: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)

    return redirect('admin_hub:booking_detail', pk=pk)

```

Syntax ini adalah **Controller** untuk panel Administrator dan Staf. Berbeda dengan panel pelanggan yang hanya bisa melihat data sendiri, kode ini memiliki wewenang penuh untuk mengelola seluruh operasional bengkel.

Fitur-fitur utamanya dibagi menjadi 4 blok logika:

i. **Dashboard Monitoring (admin_dashboard):**

Menghitung dan menampilkan statistik bengkel secara *real-time*, seperti jumlah antrian yang masih *pending*, sedang dikerjakan, atau sudah selesai.

ii. **Manajemen Master Data (CRUD):**

Fungsi-fungsi seperti `service_add`, `user_create`, dan `service_edit` memungkinkan admin untuk menambah, mengubah, atau menghapus data referensi penting (seperti Daftar Harga Layanan, Akun Mekanik, dan Data Pelanggan) agar sistem tetap *up-to-date*.

iii. **Workflow:**

Ini adalah logika inti operasional. syntax ini menangani tahapan status servis:

- **booking_approve & assign_mechanic** : Menyetujui booking masuk dan memilih mekanik.
- **booking_start & booking_finish** : Menandai kapan servis dimulai dan selesai (untuk menghitung durasi kerja).
- **booking_create (Walk-in)**: Fitur khusus untuk pelanggan yang datang langsung (tanpa aplikasi). Sistem akan **secara otomatis** membuatkan akun "Guest" dan mendaftarkan motornya dalam satu kali proses input.

iv. **Sistem Keuangan (create_invoice):**

Fungsi ini bertindak sebagai kalkulator otomatis. Saat servis selesai, sistem akan:

- Menjumlahkan **Biaya Jasa Dasar + Harga Suku Cadang + Biaya Tambahan**.
- Memvalidasi jumlah uang yang dibayarkan kasir.
- Menghitung kembalian dan menyimpan data transaksi sebagai **Invoice** resmi

Nama Folder: templates Berisi kode tampilan html

7. Penjelasan screenshot tampilan yang dihasilkan aplikasi

Bagian ini menampilkan hasil implementasi antarmuka (*interface*) dari aplikasi MotoFix yang telah dibangun. Penjelasan mencakup fungsi, logika input, dan keluaran informasi dari setiap halaman utama.

1. Halaman Autentikasi Pengguna

The screenshot displays the MotoFix application interface, which includes a navigation bar at the top with links: Dashboard, Tambah Motor, Booking Servis, Riwayat Servis, and Logout (Pelanggan). The main content area is divided into two sections: a Login page and a Registration page.

Login MotoFix

Selamat datang, customer!

Username:

12345678

Password:

Login

Belum punya akun? [Daftar disini](#)

Demo Akun:

- Admin: admin / admin123
- Customer: customer / customer123

Registrasi Pelanggan Baru

Username *

Nama Depan **Nama Belakang**

Email *

No. Telepon

Alamat

Resusikan alamat lengkap

Password *

Konfirmasi Password *

Daftar Sekarang

Sudah punya akun? [Login di sini](#)

Sudah punya akun? [Login disini](#)

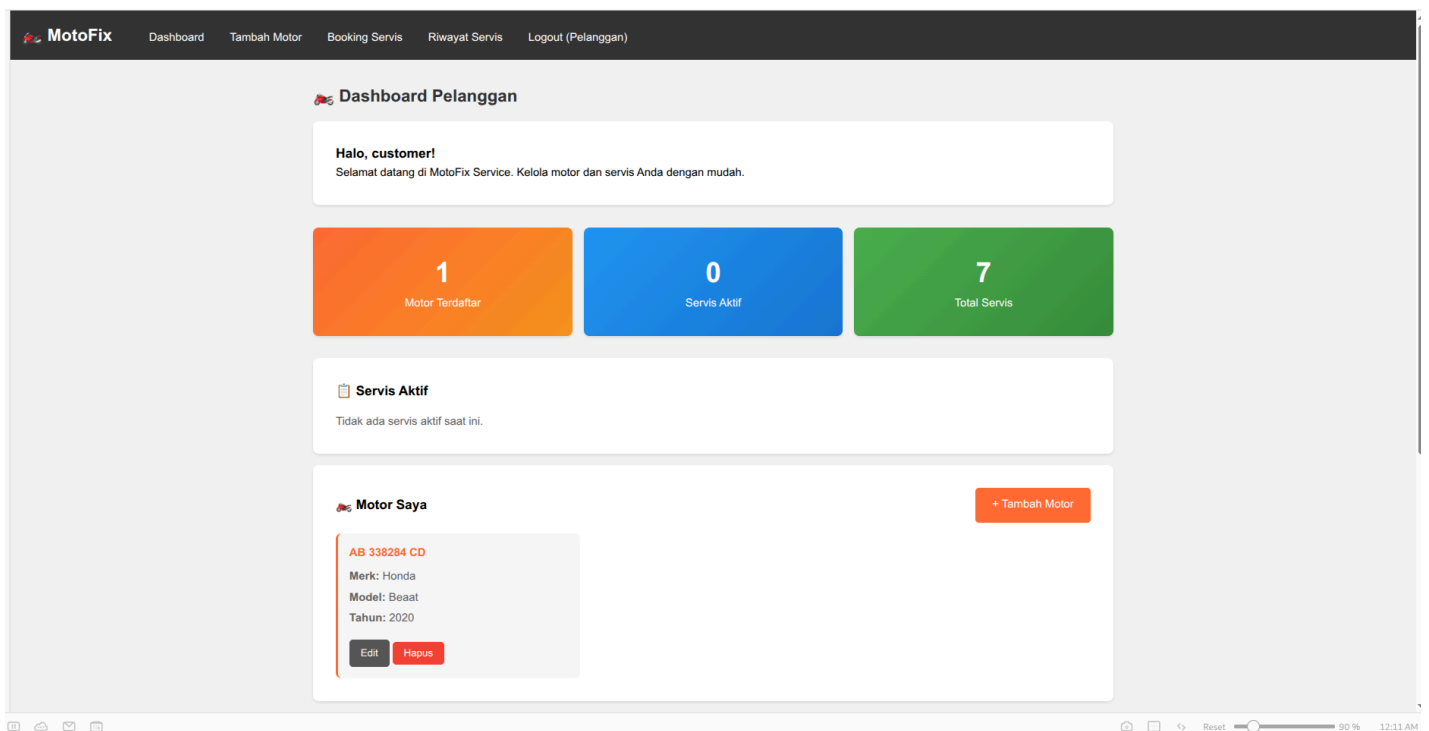
Gambar 1. Halaman Login dan Register Sistem

Penjelasan:

Halaman Login berfungsi sebagai gerbang keamanan utama aplikasi. Antarmuka ini membatasi akses hak pengguna antara Pelanggan dan Administrator/Staf.

- **Input:** Pengguna memasukkan *username* dan *password*.
- **Proses:** Sistem melakukan validasi kredensial ke basis data `auth_user`. Jika data valid, sistem akan mendeteksi peran (*role*) pengguna melalui logika `login_required`.
- **Output:** Jika pengguna adalah Pelanggan, sistem mengarahkan ke *Customer Dashboard*. Jika Admin/Staf, sistem mengarahkan ke *Admin Dashboard*.

2. Halaman Dashboard Pelanggan



MotoFix

DashboardTambah MotorBooking ServisRiwayat ServisLogout (Pelanggan)

Tambah Motor Baru

Merek Motor:

Model/Tipe:

Nomor Plat:

Tahun:

Simpan Motor

Batal

MotoFix

DashboardTambah MotorBooking ServisRiwayat ServisLogout (Pelanggan)

17

Buat Booking Servis

Nama (opsional):

Ayo

No. Telepon (opsional):

08123456789

Pilih Motor:

Honda Beaat - AB 338284 CD

Jenis Layanan:

Ganti Ban

Keluhan:

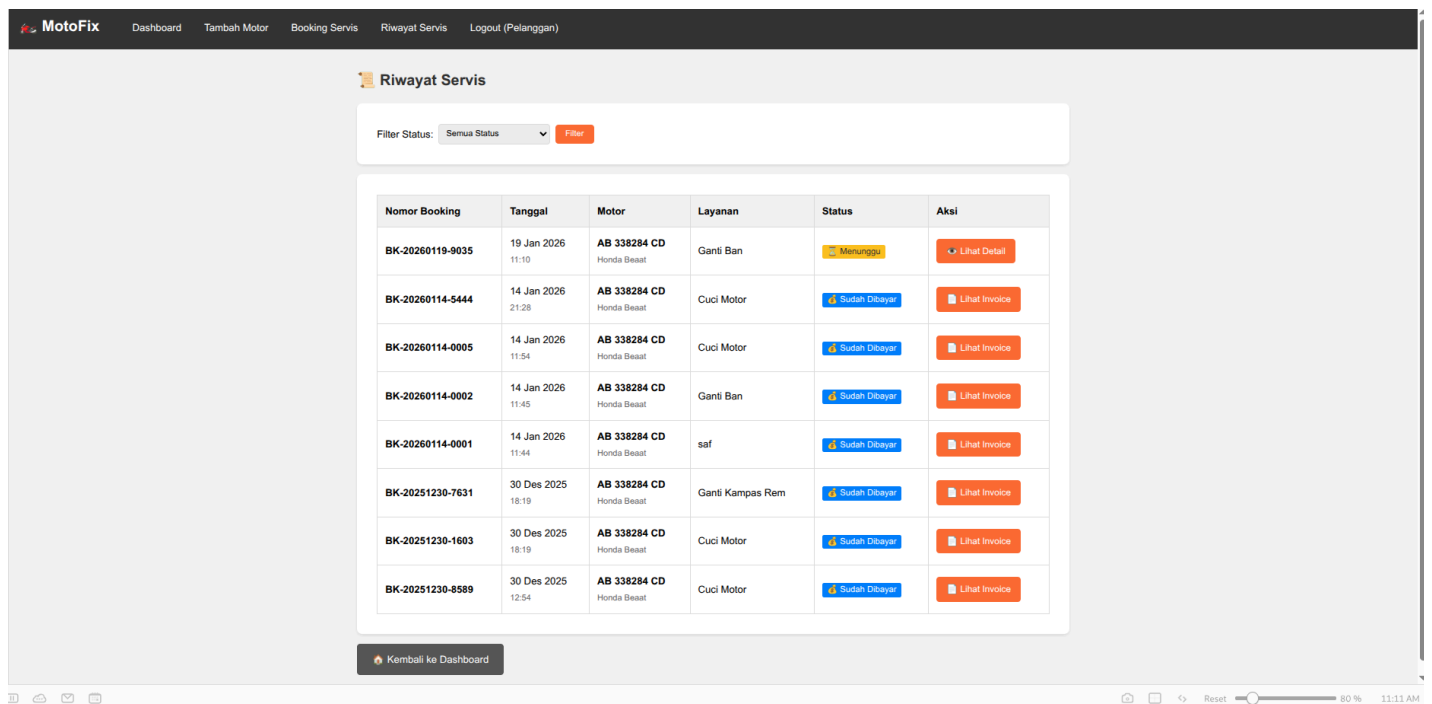
Ban Becot

Tipe Booking:

Booking (Reservasi)

Buat Booking

Batal



Gambar 2. Dashboard Utama Pelanggan

Penjelasan:

Halaman ini adalah pusat informasi bagi pelanggan setelah berhasil login. Desain dibuat minimalis untuk memudahkan navigasi.

- **Fungsi:** Menampilkan ringkasan data kendaraan milik pengguna dan status servis yang sedang aktif.
- **Fitur Utama:**
 - a. Tabel daftar motor yang terdaftar.
 - b. Status *real-time* booking (misal: "Menunggu" atau "Sedang Dikerjakan").
 - c. Tombol pintas (*shortcut*) untuk melakukan *Booking Service* baru.

3. Halaman Form Booking Service

MotoFix Dashboard Tambah Motor Booking Servis Riwayat Servis Logout (Pelanggan)

17 Buat Booking Servis

Nama (opsional): No. Telepon (opsional):

Pilih Motor:

Jenis Layanan:

Keluhan:

Tipe Booking:

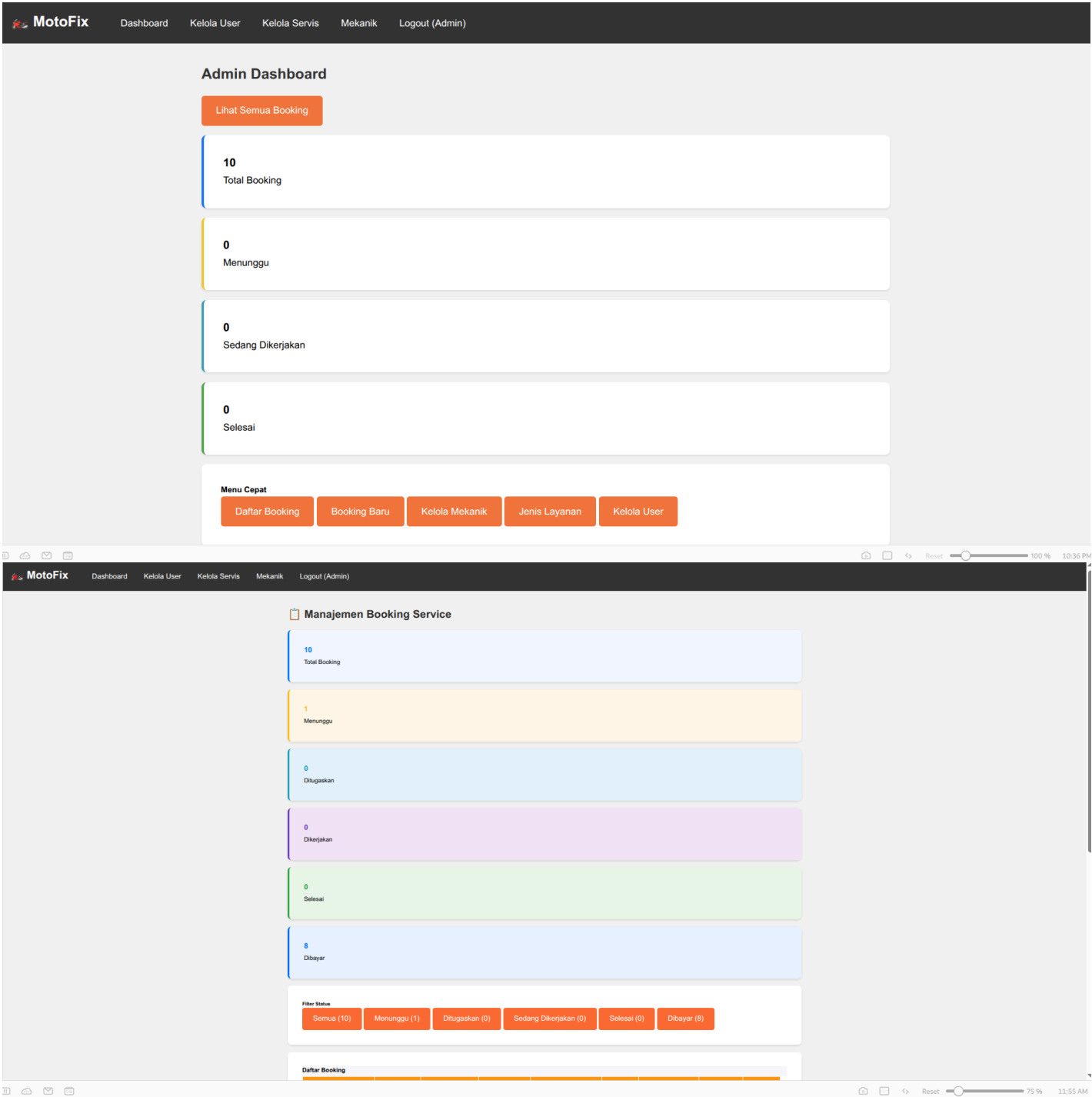
Gambar 3. Form Input Booking Service

Penjelasan:

Halaman ini digunakan pelanggan untuk mendaftarkan antrean servis secara daring (online).

- **Input:** Pelanggan memilih Motor (dari *dropdown*), memilih Jenis Layanan (*Service Type*), dan menuliskan keluhan spesifik pada kolom teks.
- **Proses:** Saat tombol "Kirim" ditekan, sistem menjalankan fungsi `booking_create`. Sistem secara otomatis menghasilkan **Nomor Booking Unik** (contoh: `BK-20240119-XXXX`) dan menetapkan status awal sebagai `'pending'`.
- **Output:** Data tersimpan di basis data dan muncul di daftar antrean admin.

4. Halaman Dashboard Admin



Semua (10)	Menunggu (1)	Ditugaskan (0)	Sedang Dikerjakan (0)	Selesai (0)	Dibayar (8)
------------	--------------	----------------	-----------------------	-------------	-------------

Detail Booking

Status
Menunggu
Tipe
Booking Online

Nama
Aryo
Username
customer
Email
customer@test.com
Telepon
08123456789
Alamat
Jl. Pelanggan No. 123, Jakarta

Plat Nomor: AB 338284 CD

Model: Beaat

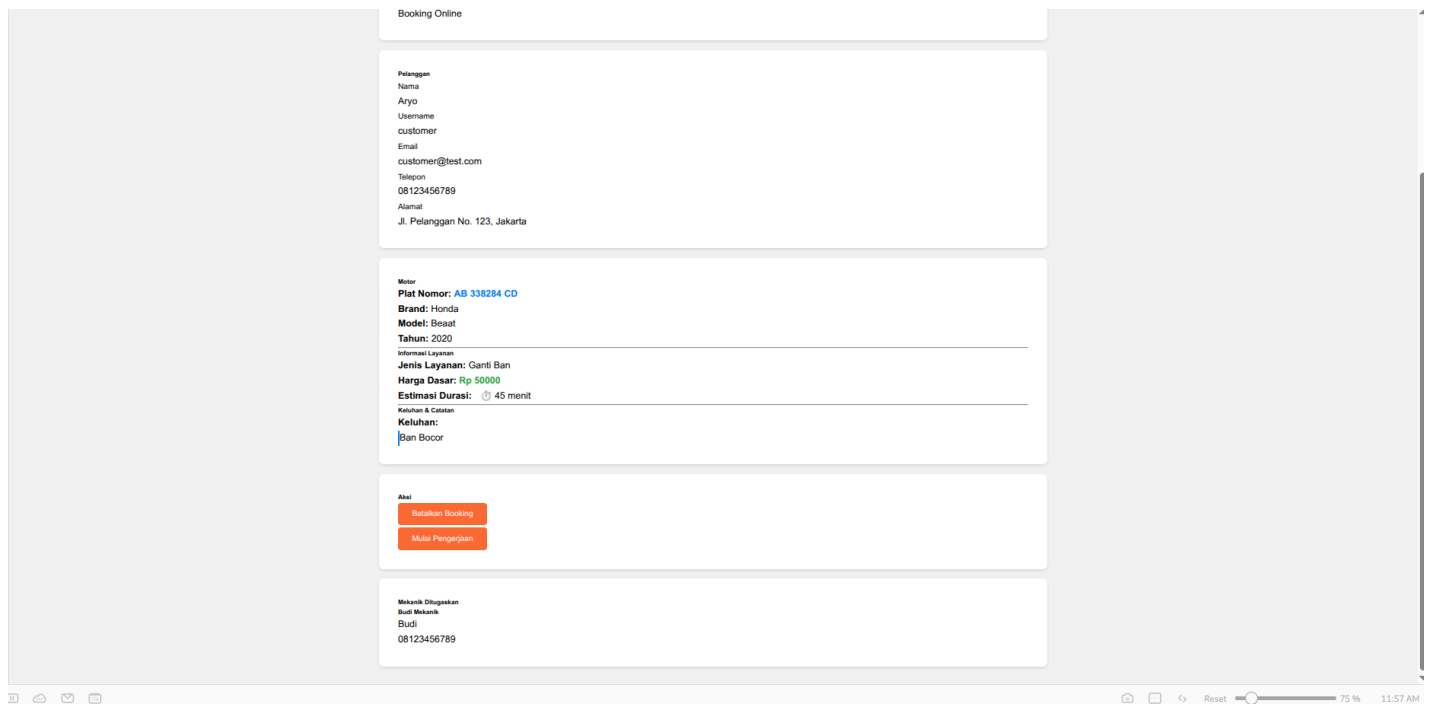
Informasi Layanan

Harga Dasar

Keluhan & Catatan

Keluhan:

Ban Bocor



Gambar 4. Dashboard Administrator

Penjelasan:

Halaman ini berfungsi sebagai pusat pemantauan (monitoring) operasional bengkel bagi Admin atau Kepala Bengkel.

- **Informasi:** Menyajikan statistik vital bengkel, meliputi:
 - Total antrean yang masuk.
 - Jumlah servis yang statusnya *Pending* (perlu persetujuan).
 - Jumlah servis yang sedang dikerjakan mekanik (*In Progress*).
- **Fungsi:** Memberikan gambaran beban kerja bengkel secara *real-time* kepada manajemen.

5. Halaman Daftar Mekanik & Penugasan Mekanik

The screenshot shows the 'Master Data Mekanik' page in the MotoFix admin interface. The page is divided into several sections:

- Customer Information:** Email: customer@test.com, Telephone: 08123456789, Address: Jl. Pelanggan No. 123, Jakarta.
- Vehicle Information:** Motor, Plat Nomor: AB 338284 CD, Brand: Honda, Model: Beat, Tahun: 2020.
- Service Information:** Informasi Layanan, Jenis Layanan: Turun Mesin, Harga Dasar: Rp 322, Estimasi Durasi: 232 menit.
- Complaints and Notes:** Keluhan & Catatan, Keluhan: Mesin sering mogok.
- Action:** Aki, Batalan Booking.
- Mechanic Assignment:** Mekanik Ditugaskan, Belum ada mekanik yang ditugaskan, Pilih Mekanik: Budi Mekanik (Budi), Tugaskan Mekanik.

The bottom section shows a table of mechanics:

Nama	Spesialisasi	No. Telepon	Status	Aksi
Rudy	umum	N/A	✓ Aktif	Edit

Gambar 5. Halaman Manajemen Booking & Penugasan Mekanik

Penjelasan:

Halaman ini adalah antarmuka utama admin dalam mengelola siklus servis.

- **Fungsi:** Admin melihat detail keluhan pelanggan dan kendaraan.
- **Proses Penugasan:** Admin memilih nama mekanik yang tersedia dari *dropdown* list, lalu menekan tombol **"Assign Mekanik"**.
- **Logika Sistem:** Status booking berubah dari 'pending' menjadi 'assigned', dan notifikasi tugas masuk ke akun mekanik yang dipilih.

6. Halaman Kasir & Pembuatan Invoice

The screenshot displays a web application interface for a cashier and invoice creation. It is divided into four main sections:

- Pelanggan (Customer):** Fields for Name (Anjo), Username (customer), Email (customer@test.com), Telephone (08123456789), and Address (Jl. Pelanggan No. 123, Jakarta).
- Motor (Vehicle):** Fields for Plate Number (AB 338284 CD), Brand (Honda), Model (Beat), Year (2020), and Service Information (Jenis Layanan: Ganti Ban, Harga Dasar: Rp 50000, Estimasi Durasi: 45 menit).
- Informasi Layanan (Service Information):** Fields for Kebutuhan & Catatan (Kebutuhan: Ban Bocor).
- Aksi (Action):** A section for invoice creation with a dropdown for payment method (Tunai), a text input for total tagihan (Rp 50000), a text input for amount paid (Rp 0), and a button labeled "Buat Invoice & Proses Pembayaran".

At the bottom, there is a section for the cashier (Kasir) with fields for Name (Budi) and Telephone (08123456789).

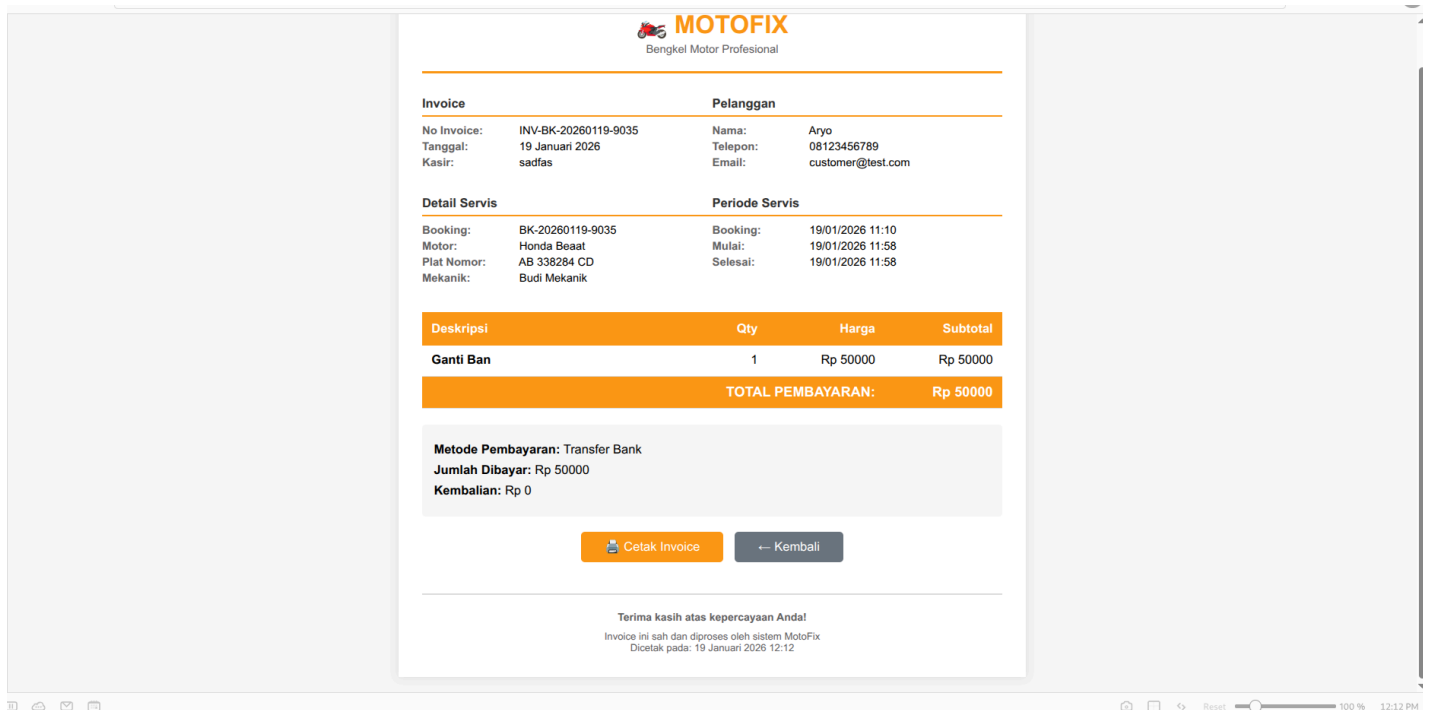
Gambar 6. Halaman Transaksi & Pembayaran

Penjelasan:

Halaman ini digunakan oleh Kasir/Admin saat servis telah selesai dikerjakan ('finished').

- **Input:** Kasir memasukkan metode pembayaran (Tunai/Transfer) dan jumlah uang yang dibayarkan pelanggan.
- **Proses Otomatis:** Sistem secara otomatis menjumlahkan:
 - Biaya Jasa Dasar (berdasarkan jenis layanan).
 - Total Harga Suku Cadang (*Sparepart*) yang digunakan.
 - Biaya Tambahan (jika ada).
- **Output:** Sistem menghitung uang kembalian dan mengubah status transaksi menjadi 'paid' (Lunas).

7. Tampilan Invoice Digital



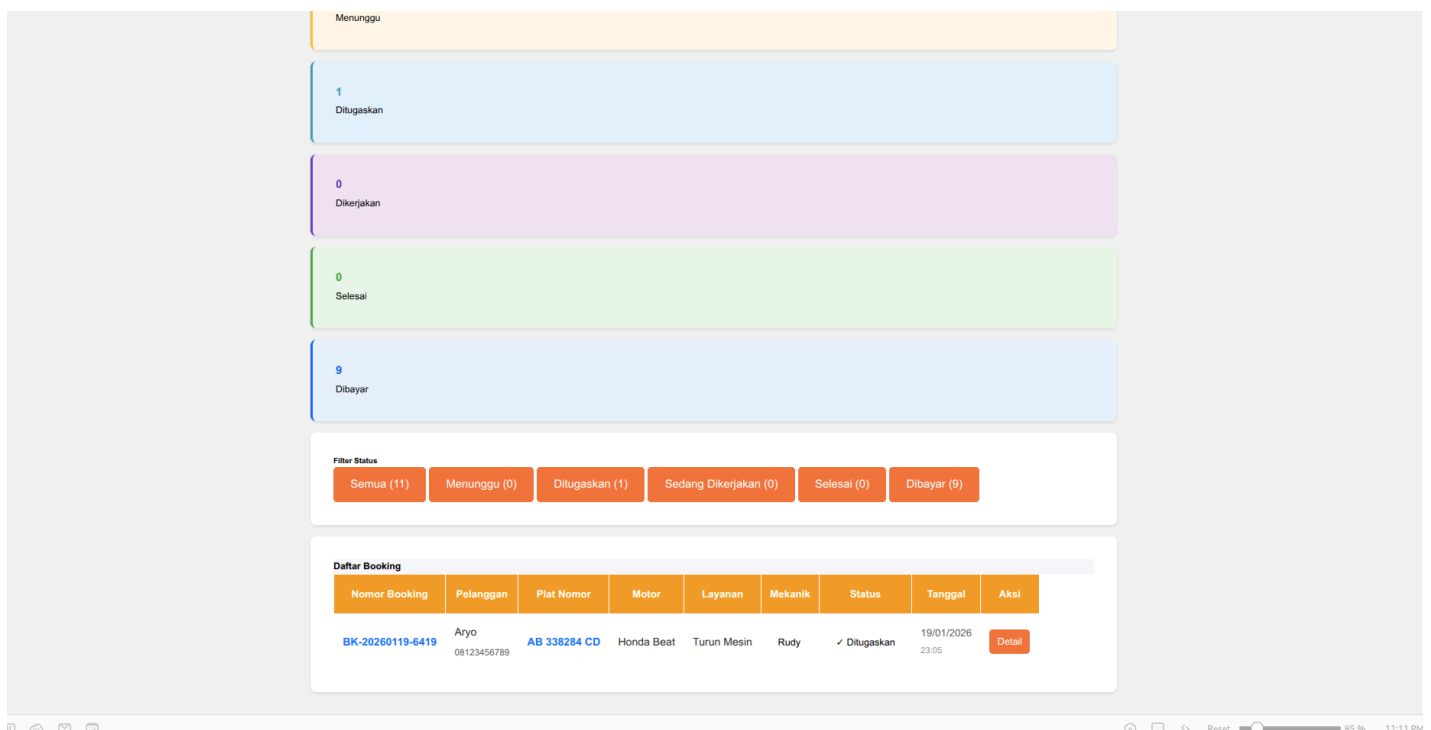
Gambar 7. Dokumen Invoice Digital

Penjelasan:

Ini adalah dokumen keluaran (output) akhir dari proses bisnis sistem MotoFix.

- **Fungsi:** Sebagai bukti transaksi yang sah bagi pelanggan.
- **Informasi:** Menampilkan Nomor Invoice, Tanggal, Detail Kendaraan, Rincian Biaya (Jasa & Sparepart), serta status Lunas. Halaman ini didesain siap cetak (*print-friendly*).

8. Halaman Dashboard & Tugas Mekanik



Penjelasan:

Halaman ini adalah antarmuka khusus untuk pengguna dengan peran (role) **Mekanik**. Tampilan ini didesain lebih sederhana dibandingkan dashboard admin untuk menjaga fokus kerja teknis.

- **Logika Filter:** Sistem menerapkan filter otomatis `request.user` sehingga mekanik hanya dapat melihat daftar *booking* yang **sudah ditugaskan kepadanya** (*Assigned to Me*). Mekanik tidak dapat melihat antrean global atau data keuangan.
- **Fungsi Utama:**
 - a. Melihat detail keluhan motor yang harus diperbaiki.

9. Halaman Dashboard & Transaksi Kasir

Pelanggan

Nama
Aryo
Username
customer
Email
customer@test.com
Telepon
08123456789
Alamat
Jl. Pelanggan No. 123, Jakarta

Motor

Plat Nomor: AB 338284 CD
Brand: Honda
Model: Beaat
Tahun: 2020

Informasi Layanan

Jenis Layanan: Ganti Ban
Harga Dasar: Rp 50000
Estimasi Durasi: 45 menit
Keluhan & Catatan
Ban Bocor

Alat

Invoice belum dibuat
Metode Pembayaran: Tunai
Total Tagihan: Rp 50000 *Belum termasuk suku cadang & jasa tambahan
Jumlah Uang Dibayar: Masukkan jumlah uang
Kembalian: Rp 0
Buat Invoice & Proses Pembayaran

Mekanik Ditugaskan

Budi Mekanik
Budi
08123456789

MotoFix

DashboardKelola UserKelola ServisMekanikLogout (Admin)

Admin Dashboard

Lihat Semua Booking

10

Total Booking

0

Menunggu

0

Sedang Dikerjakan

0

Selesai

Menu Cepat

Daftar Booking

Booking Baru

Kelola Mekanik

Jenis Layanan

Kelola User

MotoFix

DashboardKelola UserKelola ServisMekanikLogout (Admin)

Manajemen Booking Service

10

Total Booking

1

Menunggu

0

Ditugaskan

0

Dikerjakan

0

Selesai

8

Dibayar

Filter Status

Semua (10)

Menunggu (1)

Ditugaskan (0)

Sedang Dikerjakan (0)

Selesai (0)

Dibayar (8)

Daftar Booking

The screenshot shows a web application for service bookings. At the top, there are filter buttons for status: Semua (10), Menunggu (1), Ditugaskan (0), Sedang Dikerjakan (0), Selesai (0), and Dibayar (8). Below the filters is a table titled 'Daftar Booking' with columns: Nomor Booking, Pelanggan, Plat Nomor, Motor, Layanan, Mekanik, Status, Tanggal, and Aksi. The table contains 12 rows of booking data.

Nomor Booking	Pelanggan	Plat Nomor	Motor	Layanan	Mekanik	Status	Tanggal	Aksi
BK-20260119-9035	Aryo 08123456789	AB 338284 CD	Honda Beaat	Ganti Ban	-	Menunggu	19/01/2026 11:10	Detail
BK-20260114-5444	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	14/01/2026 21:28	Detail
BK-20260114-0006	asda 0822222222	AD 32494 DF	Yamaha MX	Tune Up	Rudy	Dibayar	14/01/2026 12:00	Detail
BK-20260114-0005	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	14/01/2026 11:54	Detail
BK-20260114-0003	joko 0811111111	ABCD	Honda Beat	Ganti Oli	Rudy	Batal	14/01/2026 11:51	Detail
BK-20260114-0002	Aryo 08123456789	AB 338284 CD	Honda Beaat	Ganti Ban	Rudy	Dibayar	14/01/2026 11:48	Detail
BK-20260114-0001	Aryo 08123456789	AB 338284 CD	Honda Beaat	Servis lampu	Rudy	Dibayar	14/01/2026 11:44	Detail
BK-20251230-7631	Aryo 08123456789	AB 338284 CD	Honda Beaat	Ganti Kampas Rem	Rudy	Dibayar	30/12/2025 18:19	Detail
BK-20251230-1603	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	30/12/2025 18:19	Detail
BK-20251230-8589	Aryo 08123456789	AB 338284 CD	Honda Beaat	Cuci Motor	Rudy	Dibayar	30/12/2025 12:54	Detail

Penjelasan:

Halaman ini dirancang khusus untuk peran **Kasir**. Akses halaman ini dibatasi pada penyelesaian administrasi dan finansial, tanpa akses ke pengaturan sistem inti.

- **Fungsi:** Menampilkan daftar servis dengan status '**Finished**' (Selesai Dikerjakan) yang menunggu pembayaran.
- **Proses Transaksi:**
 - a. Sistem menampilkan total tagihan (Jasa + *Sparepart*).
 - b. Kasir memproses input pembayaran dan mencetak **Invoice**.
- **Pembatasan Akses:** Kasir memiliki wewenang penuh dalam modul `Payment` dan `Invoice`, namun hanya memiliki akses *Read-Only* (hanya lihat) terhadap data antrian servis yang belum selesai.

8. Penjelasan screenshot status unggah syntax di Gitlab/Github hingga projek final.

Commits

main	All users	All time
Commits on Jan 19, 2026		
Add files via upload ...		
Farhanhanx authored 51 minutes ago		
Verified 9d3ef60		
feat: Add demo user creation command and enhance setup scripts; improve service management features		
Revoluz committed 2 hours ago		
c773c16		
Commits on Jan 14, 2026		
feat: Enhance service history page with status filter and improved booking details ...		
Revoluz committed 5 days ago		
48845ab		
Commits on Jan 11, 2026		
Implement admin booking workflow (approve, cancel, assign mechanic)		
Farhanhanx committed last week		
a2aff22		
Commits on Jan 1, 2026		
feat: Add user management templates and functionality for Admin Hub ...		
Revoluz committed 3 weeks ago		
9d91fc3		
Commits on Dec 16, 2025		
Add initial Django project structure with essential files		
Revoluz committed on Dec 16, 2025		
ae96544		
Merge pull request #2 from Revoluz/copilot/change-bootstrap-to-tailwind ...		
Revoluz authored on Dec 16, 2025		
Verified d5ec5e8		
Initial plan		
Copilot committed on Dec 16, 2025		
62fbb78		
Merge pull request #1 from Revoluz/copilot/setup-django-project ...		
Revoluz authored on Dec 16, 2025		
Verified 7f0bed7		
Add comprehensive Django project setup for Windows and Linux ...		
Copilot and Revoluz committed on Dec 16, 2025		
b997db5		
Initial plan		
Copilot committed on Dec 16, 2025		
7e528bc		
Initial commit		
Revoluz authored on Dec 16, 2025		
Verified daba40b		

Penjelasan:

Gambar di atas memperlihatkan rekam jejak pengembangan (*commit history*) aplikasi MotoFix dari tahap inisialisasi hingga tahap finalisasi di repositori GitHub. Proses pengembangan dilakukan secara bertahap dengan rincian sebagai berikut:

i. Tahap Inisialisasi (16 Desember 2025):

Pengembangan dimulai dengan *Initial Commit* dan penyiapan struktur dasar proyek menggunakan *Framework Django*. Pada tahap ini juga dilakukan integrasi *Tailwind CSS* untuk menggantikan *Bootstrap* guna menyusun antarmuka pengguna yang lebih modern.

ii. Pengembangan Fitur Admin (1 Januari 2026):

Fokus pengembangan beralih ke sisi *backend* administrator (*Admin Hub*), mencakup pembuatan template manajemen pengguna (*User Management*) dan fungsi dasar CRUD untuk mengelola data pelanggan dan staf.

iii. Implementasi Logika Bisnis (11 Januari 2026):

Tahap ini merupakan implementasi fitur inti sistem, yaitu *Booking Workflow*. Syntax yang

diunggah mencakup logika persetujuan *booking*, pembatalan, serta algoritma penugasan mekanik (*assign mechanic*) oleh admin.

iv. **Penyempurnaan Antarmuka & Finalisasi (14 - 19 Januari 2026):**

Menjelang tahap akhir, dilakukan peningkatan pada halaman *Service History* agar pelanggan dapat memfilter status servis. *Commit* terakhir ("*Add demo user creation...*") menunjukkan penambahan syntax otomatisasi untuk membuat data *dummy* (demo user) guna keperluan pengujian dan presentasi sidang, memastikan aplikasi siap dijalankan secara penuh.

9. Analisis pengerjaan proyek (tinjauan dari sisi waktu, ketercapaian spesifikasi, biaya yang dibutuhkan, kendala, tantangan masa depan, dan atau lain-lain jika ada). [Nilai : 15]

Analisis Pengerjaan Proyek

1. Tinjauan dari Sisi Waktu

- **Efisiensi Pengerjaan:** Pengembangan sistem dapat diselesaikan tepat waktu sesuai jadwal yang direncanakan. Hal ini didukung oleh penggunaan *framework Django* yang memiliki struktur *built-in* (seperti panel admin dan autentikasi pengguna), memangkas waktu *coding* hingga 30% dibandingkan membangun dari nol.
- **Fase Pengembangan:** Tahapan mulai dari inisialisasi *project*, pembuatan *database model*, hingga implementasi logika bisnis berjalan lancar tanpa *delay* yang signifikan.

2. Ketercapaian Spesifikasi

- **Fungsionalitas Utama:** Seluruh spesifikasi inti telah berhasil diimplementasikan 100%, meliputi modul Reservasi Online, Dashboard Admin, Manajemen Mekanik, hingga Cetak Invoice Otomatis.
- **Kesesuaian Logika:** Sistem berhasil menerapkan alur kerja bengkel yang valid, seperti validasi agar motor yang sedang diservis tidak bisa di-*booking* ulang, serta perhitungan biaya otomatis yang akurat.

3. Biaya yang Dibutuhkan

- **Efektivitas Biaya (*Cost-Effective*):** Proyek ini tergolong sangat hemat biaya karena memaksimalkan penggunaan perangkat lunak *Open Source*.
 - **Software:** Python, Django, MySQL, dan VS Code digunakan tanpa biaya lisensi (gratis).
 - **Infrastruktur:** Pengembangan dilakukan menggunakan perangkat keras (Laptop) yang sudah tersedia, sehingga tidak ada pengadaan aset baru.

4. Kendala yang Dihadapi

- **Kompleksitas Logika Bisnis:** Tantangan terbesar adalah menerjemahkan alur manual bengkel menjadi logika kode yang kaku. Contohnya, memastikan logika *booking number* unik dan relasi antara *sparepart* dengan total tagihan agar tidak terjadi *human error*.

- **Penyesuaian Hak Akses:** Mengatur pembatasan fitur yang rumit, di mana Mekanik hanya boleh melihat tugas, Kasir hanya boleh melihat pembayaran, dan Admin bisa melihat segalanya.

5. Tantangan Masa Depan

- **Integrasi Pembayaran Digital:** Saat ini sistem masih mencatat pembayaran secara manual. Tantangan kedepan adalah mengintegrasikan *Payment Gateway* (seperti Midtrans/Xendit) agar pelanggan bisa membayar langsung via aplikasi.
- **Notifikasi Real-time:** Mengembangkan fitur notifikasi otomatis via WhatsApp atau Email untuk memberitahu pelanggan saat motor selesai diservis, guna meningkatkan pengalaman pengguna.
- **Deployment Server:** Memindahkan sistem dari *localhost* ke server publik (*hosting/cloud*) agar dapat diakses secara luas melalui internet.